Chapter 5

Simulation

Learning Objectives

By the end of this chapter, students will be able to:

1. Explain why simulation is needed when closed-form valuations are unavailable, and recognize the trade-off between flexibility and computational cost.
2. Articulate the connection between Monte Carlo simulation and numerical integration, and explain why simulation wins in high-dimensional problems.
3. Generate pseudorandom numbers and apply the inverse-CDF method to sample from any continuous distribution.
4. Write a Python Monte Carlo simulation for a valuation problem using NumPy, SciPy, and pandas.
5. Apply antithetic variates, moment matching, and control variates to reduce simulation variance.
6. Simulate correlated multivariate normal random vectors using the Cholesky decomposition.
7. Use a Gaussian copula to model dependence between non-normal random variables.
8. Apply common random numbers for sensitivity analysis and optimization.

5.1 Introduction

In Chapters 2 through 4 we valued cash flows when their patterns could be expressed in closed form: flat annuities, growing annuities, mean-reverting annuities, Treasury bonds, defaultable bonds with constant hazard rates. The General Valuation Equation, paired with a discount curve, sufficed to produce exact prices.

But finance is full of problems where no closed-form solution exists. Consider:

- A portfolio of 50 defaultable loans whose defaults are *correlated* through a common business-cycle factor. The expected loss is easy; the σ/Standard Deviation is not.
- An exotic option whose payoff depends on the *path* the underlying takes over its lifetime, not just its terminal value.
- A natural-resource project whose cash flows depend on a commodity price that follows a stochastic process with both mean reversion and jumps.
- A pension obligation whose present value depends on uncertain longevity, inflation, and salary growth, all correlated with each other and with market returns.

Each problem shares a common structure: a known model of randomness, a deterministic mapping from random variables to cash flows, and a question about expected value (or risk). When the mapping is simple enough, analytic formulas work. When it isn't, simulation is the universal fallback. With modern computing, simulation has become fast, accurate, and accessible, it is the workhorse of modern financial valuation.

This chapter has four goals:

- **1. Establish the conceptual foundation.** Simulation is *Monte Carlo integration*: a way to estimate expectations by averaging random draws. Understanding this connection makes everything else in the chapter intuitive.
- **2. Build practical Python skills.** Every method is illustrated with executable Python code using NumPy, SciPy, matplotlib, and pandas. Students should be able to copy the code, run it, and modify it.
- **3. Develop the variance-reduction toolkit.** Antithetic variates, moment matching, and control variates can dramatically improve simulation accuracy at zero extra computational cost.
- **4. Extend to multivariate problems.** The Cholesky decomposition and Gaussian copula let us simulate correlated risks, the foundation of modern portfolio and credit risk modeling.

Python Environment

All code examples assume Python 3.10+ with NumPy, SciPy, matplotlib, and pandas installed. Standard imports throughout are shown below.

Standard imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats, integrate
from scipy.stats import norm
```

5.2 Simulation as Numerical Integration

Before we write any simulation code, we need a mental model of what simulation actually does. The model is simple:

Monte Carlo simulation is a method for computing integrals. Once you see this, every technique in the chapter, variance reduction, multivariate sampling, common random numbers, becomes intuitive.

5.2.1 The Integral Behind Every Valuation

Suppose a cash flow C depends on a random variable X through a function g: $C = g(X)$. If X has probability density $f(x)$, the expected cash flow is the integral:

$$\mathbb{E}[C] = \mathbb{E}[g(X)] = \int g(x) \cdot f(x)\,\mathrm{d}x$$

Every Monte Carlo simulation is a numerical method for evaluating this integral. Throughout this chapter we work in a single period, so discounting is folded directly into $g(x)$: when a cash flow arrives one period from today, $g(x)$ already denotes its present value, and the valuation is simply $V_0 = \mathbb{E}[C] = \mathbb{E}[g(X)]$.

Two ways to evaluate this integral:

- **Deterministic numerical integration** (Simpson's rule, Gauss quadrature, etc.) discretizes the integration domain into a regular grid and applies a weighted sum.
- **Monte Carlo integration** randomly samples $X_1, X_2, \ldots, X_N$ from $f(x)$ and approximates the expectation by the sample mean:

$$\mathbb{E}[g(X)] \approx \frac{1}{N} \sum_{i=1}^{N} g(X_i)$$

This is the **Monte Carlo estimator.** By the Law of Large Numbers, it converges to the true expectation as $N \to \infty$.

The distribution of $\mathbb{E}[g(X)]$ also converges in distribution to the normal density function, according to the Central Limit Theorem. This is an especially valuable observation since we can use this to estimate the likely maximum error of $\mathbb{E}[g(X)]$, which is normally taken as 2-3 times the standard error of the simulation.

The standard error decreases as $\sigma/\sqrt{N}$, where $\sigma^2 = Var[g(X)]$.

5.2.2 Simpson's Rule vs. Monte Carlo: A Concrete Comparison

Suppose a project's cash flow next period, X, is uncertain: it is normally distributed with mean $\mu = \$100$ (million) and standard deviation $\sigma = \$20$ (million). A bonus arrangement pays out the amount by which the cash flow exceeds its own mean, and nothing if it falls short or lands exactly on it. What is that bonus worth, on average, before the cash flow is realized?

The figure below shows both halves of the question on the same picture. The blue curve is the probability density of X, where the cash flow is actually likely to land. The red line is the payout rule itself, flat at \$0 up to μ and then rising dollar-for-dollar above it. The darker shaded region is the half of that density where the bonus is actually earned.

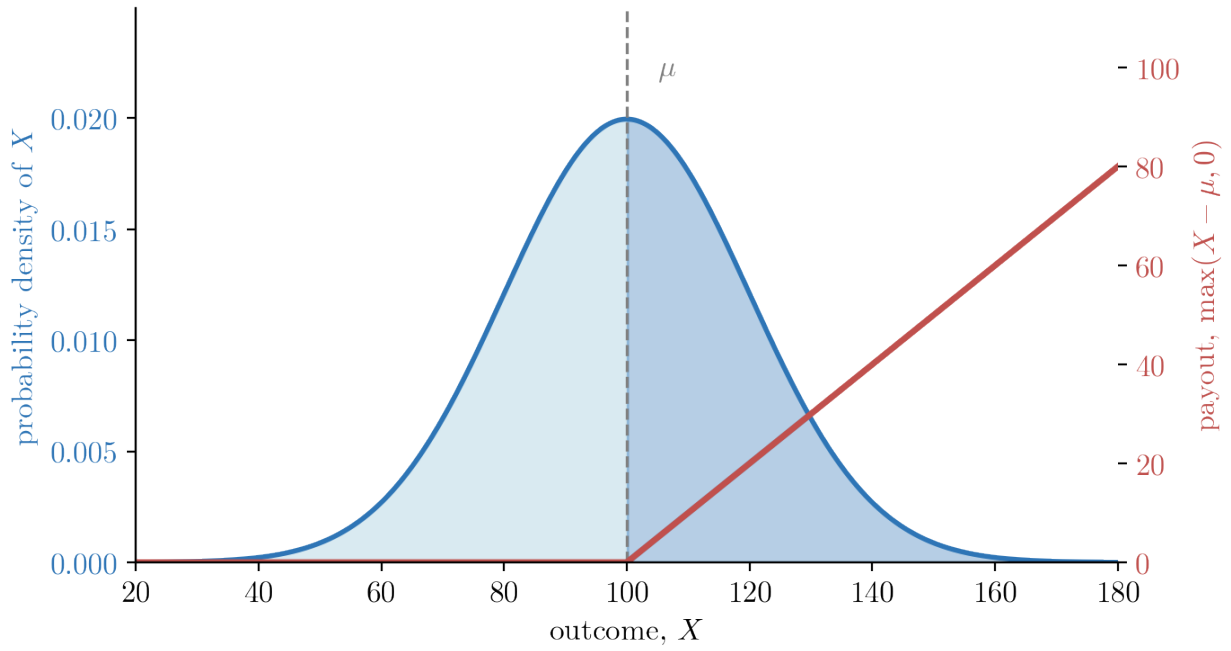


Figure 5.1: *The payout rule (red, right axis) against the probability density of the cash flow X (blue, left axis). The darker shading marks the half of the distribution where the bonus is earned.*

This payout is a function $g(X)$ of the cash flow, exactly the setup from Section 5.2.1, so its value today is $V_0 = \mathbb{E}[g(X)]$, the average payout, weighted by how likely each outcome is. This particular expectation happens to have a clean closed form. Writing $X - \mu = \sigma Z$ for a standard normal Z,

$$\mathbb{E}[\max(X - \mu, 0)] = \sigma\,\mathbb{E}[\max(Z, 0)] = \frac{\sigma}{\sqrt{2\pi}}$$

since $\mathbb{E}[\max(Z, 0)]$ is just the area under the standard normal density to the right of zero, weighted by z, which integrates to $1/\sqrt{2\pi}$. No cumulative normal table and no convexity adjustment are needed. We will pretend we don't have this formula, value the bonus two ways, and check both against it.

Method 1: Simpson's rule. Express the expected payoff as an integral over X itself, which is normal with mean μ and standard deviation σ. The payoff is zero below μ, so discretize only the region $x \geq \mu$ and apply Simpson's rule:

Simpson's rule

```
from scipy.integrate import simpson

mu, sigma = 100, 20

# Integration domain: mu to mu + 8 sigma (effectively infinity);
# the payoff is zero below mu, so there is nothing to integrate there
n_grid = 101
```

```
x_grid = np.linspace(mu, mu + 8*sigma, n_grid)
integrand = (x_grid - mu) * norm.pdf(x_grid, mu, sigma)
simpson_price = simpson(integrand, x=x_grid)
print(f"Simpson's rule (n=101): {simpson_price:.6f}")
```

Output

```
Simpson's rule (n=101): 7.978851
```

The closed form is $\sigma/\sqrt{2\pi} = 7.978846$, so 101 evaluations already match it to five decimal places. Increasing to $n = 1001$ recovers all six: 7.978846.

Method 2: Monte Carlo. Sample values of X, evaluate the payoff, and average:

Monte Carlo

```
np.random.seed(42)
results = []
for N in [100, 1000, 10000, 100000]:
    X = np.random.normal(mu, sigma, size=N)
    payoffs = np.maximum(X - mu, 0)
    mc_price = payoffs.mean()
    se = payoffs.std() / np.sqrt(N)
    results.append((N, mc_price, se))
    print(f"N={N:>6}: price={mc_price:.6f}, SE={se:.6f}")
```

Output

```
N=   100: price=6.189556, SE=0.951165
N=  1000: price=8.286480, SE=0.378721
N= 10000: price=7.893409, SE=0.116299
N=100000: price=8.009631, SE=0.036978
```

Before comparing how well the two methods agree, it helps to see how differently they get there; the figure below shows the actual mechanics side by side.

Simpson's rule (the left panel of Figure 5.2) evaluates the integrand $f(x) = (x - \mu)\,\phi(x)$, the payoff times the density of X, on a fixed, evenly-spaced grid $x_0, x_1, \ldots, x_n$ with spacing h over the region $x \geq \mu$ where the payoff is positive, shown here with 13 of the true 101 points for legibility, then fits a parabola through each consecutive triple of points and sums the exact area under each parabola:

$$\int f(x)\,\mathrm{d}x \approx \frac{h}{3}\Big[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 4f(x_{n-1}) + f(x_n)\Big]$$

the alternating $4, 2, 4, 2, \ldots$ weights are exactly what makes each parabolic segment's area exact rather than a linear approximation between points; this is what `scipy.integrate.simpson`

computes internally.

Monte Carlo (the right panel of Figure 5.2) instead draws random values of X and averages the payoff directly. Recall the general estimator from Section 5.2.1, $\mathbb{E}[g(X)] \approx \frac{1}{N}\sum g(X_i)$: here $g(X) = \max(X - \mu, 0)$, the excess above the mean when there is one, and zero otherwise. Averaging that payoff over N draws is exactly:

$$\hat{V}_0 = \frac{1}{N}\sum_{i=1}^{N} \max(X_i - \mu, 0)$$

This is precisely what the code computes, via `payoffs.mean()`. It is also exactly what the figure's shaded region represents. It is not a separate trick. It is the same sample-mean estimator introduced earlier in the chapter, applied here to one particular g. One method is a deterministic sum over a known curve. The other is a random sample average that only converges to that same curve as N grows.

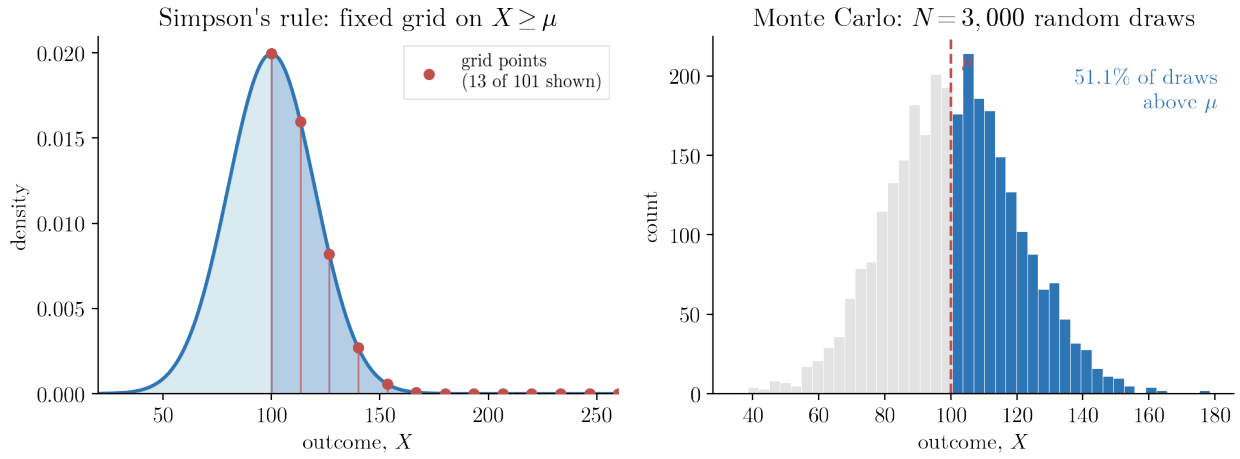


Figure 5.2: *How each method computes the same price: Simpson's rule integrates a fixed grid over the payoff-weighted density (left); Monte Carlo averages the payoff over random draws of X (right).*

The table above makes the convergence point numerically. The figure below makes the same point visually, and side by side. Simpson's rule is plotted as its own convergence path as the grid size n grows from 5 to 101: it is within 2% of the true value by $n = 11$, matches to three decimal places by $n = 51$, and to six by $n = 1001$. Monte Carlo is plotted as its 95% confidence interval as the sample size N grows from 100 to 100,000. The interval visibly shrinks as N grows, but only slowly: going from $N = 100$ to $N = 100{,}000$, a thousand-fold increase in evaluations, narrows the interval by roughly a factor of 30, matching the $1/\sqrt{N}$ scaling. Simpson's rule needed only 101 evaluations to match Monte Carlo's 100,000-sample precision.

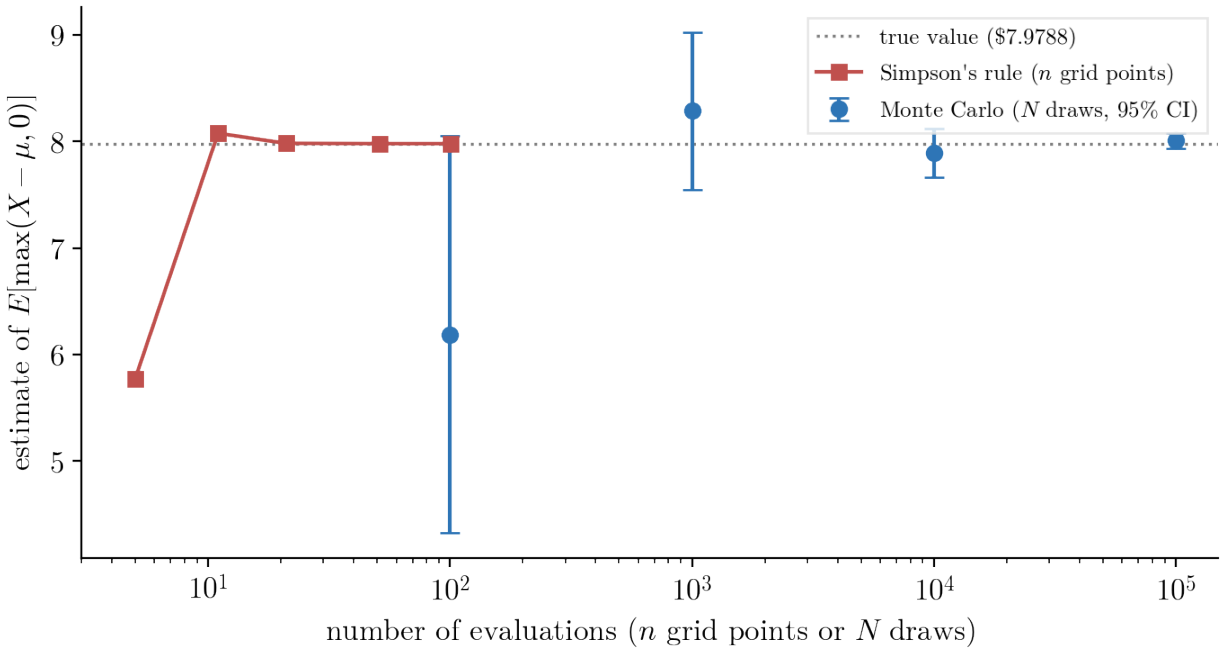


Figure 5.3: *Price estimate vs. number of evaluations for both methods. Simpson's rule (red) converges within a handful of grid points; Monte Carlo (blue, with 95% CI) only tightens onto the true price gradually as N grows into the tens of thousands.*

Comparison. Simpson's rule with 101 evaluations achieves five-decimal accuracy. Monte Carlo with 100,000 samples, a thousand times more evaluations, has standard error 0.037, meaning the 95% confidence interval is roughly ± 0.07, only one-decimal accuracy. In one dimension, deterministic methods win decisively.

5.2.3 The Curse of Dimensionality (and Why Monte Carlo Wins in High Dimensions)

Now suppose the cash flow depends on d random variables: $C = g(X_1, \ldots, X_d)$. The integral becomes d-dimensional:

$$\mathbb{E}[g(X)] = \int \cdots \int g(x_1, \ldots, x_d)\, f(x_1, \ldots, x_d)\, \mathrm{d}x_1 \cdots \mathrm{d}x_d$$

If we use Simpson's rule with n grid points per dimension, the total number of evaluations is n^d:

Dimension d	Grid n^d (n=51)	Time at 1M evals/sec
1	51	0.05 ms
2	2,601	2.6 ms
3	133,000	0.13 s
5	345 million	5.7 minutes

Dimension d	Grid n^d (n=51)	Time at 1M evals/sec
10	1.2×10^{17}	380 million years

This explosion is the **curse of dimensionality.** Even modest $d = 5$ pushes us past practical limits.

Monte Carlo simulation, by contrast, has standard error $\sigma/\sqrt{N}$ *independent of dimension.* Doubling d doesn't change the number of samples needed to achieve a given accuracy. A 10-dimensional problem with 100,000 Monte Carlo samples takes the same time as a 1-dimensional problem with 100,000 samples, while Simpson's rule becomes computationally infeasible.

This explains why simulation dominates derivative pricing for path-dependent options (which depend on hundreds of intermediate prices), credit portfolio valuation (hundreds of correlated default times), and corporate cash flow valuation (multiple stochastic drivers).

Big-Picture Takeaway

- In 1D, **use deterministic integration** if you can. It is faster and more accurate.
- In low-moderate dimensions (1-2), either method works; deterministic still has an edge.
- In high dimensions (2+), **Monte Carlo is essentially the only option.** Its dimension-independent error rate is its superpower.
- The Monte Carlo error $\sigma/\sqrt{N}$ has only two levers. Increasing N is the expensive one: every halving of the error costs four times the compute. Shrinking σ is the cheap one, and it is what every variance reduction technique introduced later does. That is how Monte Carlo stays competitive on accuracy even in the low-dimensional problems where deterministic integration is available.

5.3 Random Number Generation

Monte Carlo simulation requires a source of randomness. Modern computers cannot produce truly random numbers; instead they produce **pseudorandom numbers**, deterministic sequences that pass statistical tests for randomness. NumPy's default generator is the PCG64 algorithm, which has a period of 2^{128}, produces uniform numbers on [0, 1), and passes all standard randomness tests. For practical financial work it is indistinguishable from true randomness.

Excel's own `RAND()` function uses a different algorithm: the Mersenne Twister (specifically MT19937), adopted starting with Excel 2010 to replace an older, weaker generator. Mersenne Twister has an extremely long period, $2^{19937} - 1$, and like PCG64 it passes standard randomness tests, though the two algorithms are built differently under the hood.

The seed and reproducibility. Setting a seed makes a simulation reproducible: the same seed always produces the same sequence. This is essential for debugging, testing, and reporting

reproducible research. In NumPy:

Random number generation in NumPy

```
# Set seed for reproducibility
np.random.seed(42)

# Generate 1000 uniform random numbers on [0, 1)
u = np.random.uniform(0, 1, size=1000)

# Generate 1000 standard normal random numbers
z = np.random.normal(0, 1, size=1000)

# Generate 5000 lognormal numbers with mu=4.6, sigma=0.3
ln_x = np.random.lognormal(mean=4.6, sigma=0.3, size=5000)
```

The histograms below show the three samples this code produces: flat for the uniform, the bell for the standard normal, and the right-skewed lognormal, whose long right tail is the shape to remember when we model prices and losses.

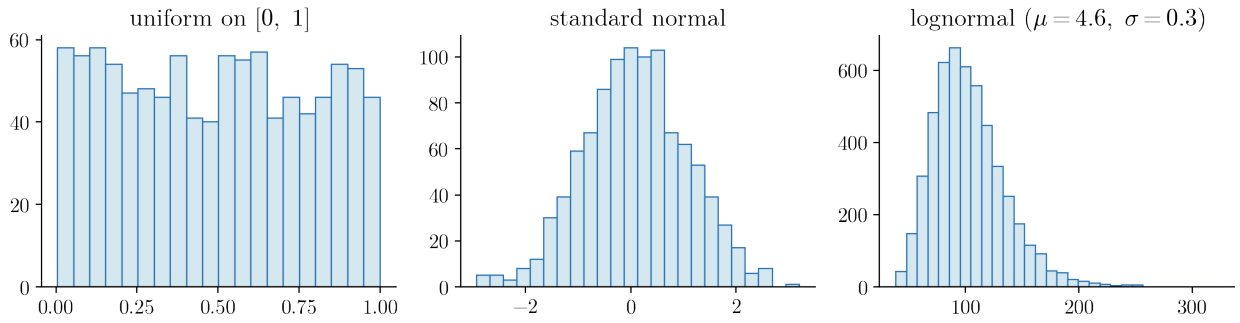


Figure 5.4: *Histograms of 1,000 uniform draws, 1,000 standard normal draws, and 5,000 lognormal draws ($\mu = 4.6$, $\sigma = 0.3$) generated by the code above.*

5.3.1 Sampling from Arbitrary Distributions: The Inverse-CDF Method

Suppose we want to sample from a continuous distribution with CDF *F(x)*. The **inverse-CDF method** (also called the **probability integral transform**) works as follows:

- **1.** Generate a uniform random number $U \sim$ *Uniform(0, 1)*.
- **2.** Compute $X = F^{-1}(U)$, the inverse CDF (quantile function) evaluated at U.

Then X has the desired distribution.

The proof has three short steps, and it is worth pausing on the middle one, since that is where students usually get stuck. First, $P(X \leq x) = P(F^{-1}(U) \leq x)$ just substitutes the definition $X = F^{-1}(U)$. Second, because F is non-decreasing, applying F to both sides of an inequality

preserves its direction. That means the event $F^{-1}(U) \le x$ is exactly the same event as $U \le F(x)$, nothing more subtle than relabeling which side of the inequality carries the transformation. Third, since U is Uniform(0,1), $P(U \le u) = u$ for any u between 0 and 1, so $P(U \le F(x)) = F(x)$ directly. Chaining the three steps gives $P(X \le x) = F(x)$, which is precisely the statement that X has CDF F.

The picture below shows why the method works, using the exponential distribution from the example that follows. The left panel is the distribution itself: the density $f(x)$, tall near zero and thin in the tail. The right panel is its CDF, which is steep exactly where the density is tall (the CDF's slope *is* the density) and flat where it is thin. The inverse-CDF method enters on the vertical axis: a uniform draw u moves horizontally to the curve, then drops down to $x = F^{-1}(u)$. Follow the three example draws $u = 0.25, 0.60, 0.90$: they land at $x \approx 5.8$, 18.3, and 46.1 years. Equal steps in u produce very unequal steps in x, because a steep stretch of CDF absorbs a lot of u-probability into a narrow band of x. That is the whole mechanism: uniform draws crowd into the regions where the density is tall, which is exactly what sampling from f means. Note also the link between the panels: the shaded area 0.60 under the density on the left *is* the height 0.60 on the CDF at the same $x = 18.3$.

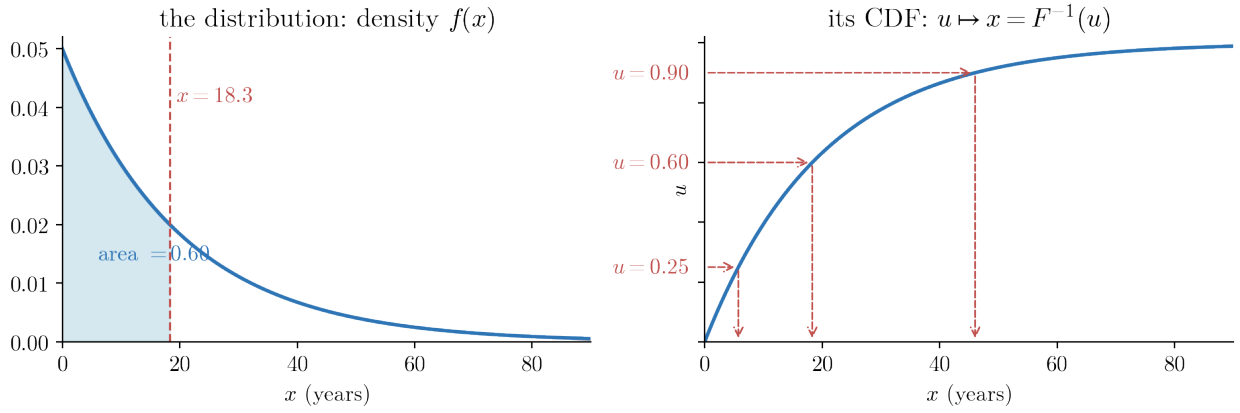


Figure 5.5: *An exponential distribution's density $f(x)$ (left) and CDF $F(x)$ (right), with three uniform draws $u = 0.25, 0.60, 0.90$ mapped through $x = F^{-1}(u)$ to $x \approx 5.8, 18.3, 46.1$ years.*

Example: exponential distribution. The exponential CDF is $F(x) = 1 - e^{-\lambda x}$, with inverse $F^{-1}(u) = -\ln(1-u)/\lambda$. Since $1-U$ is uniform whenever U is, we can use $-\ln(U)/\lambda$:

Inverse-CDF for exponential

```
# Sample default times: exponential with rate lambda = 0.05/year
lam = 0.05
N = 10000
u = np.random.uniform(0, 1, size=N)
default_times = -np.log(u) / lam
print(f"Mean: {default_times.mean():.2f} years (theoretical: {1/lam:.2f})")
print(f"Std: {default_times.std():.2f} years (theoretical: {1/lam:.2f})")
```

Output

```
Mean: 19.90 years (theoretical: 20.00)
Std: 19.92 years (theoretical: 20.00)
```

Plotting the 10,000 sampled default times against the exponential density they were drawn from confirms the method visually: the histogram traces the theoretical curve $\lambda e^{-\lambda t}$ closely.

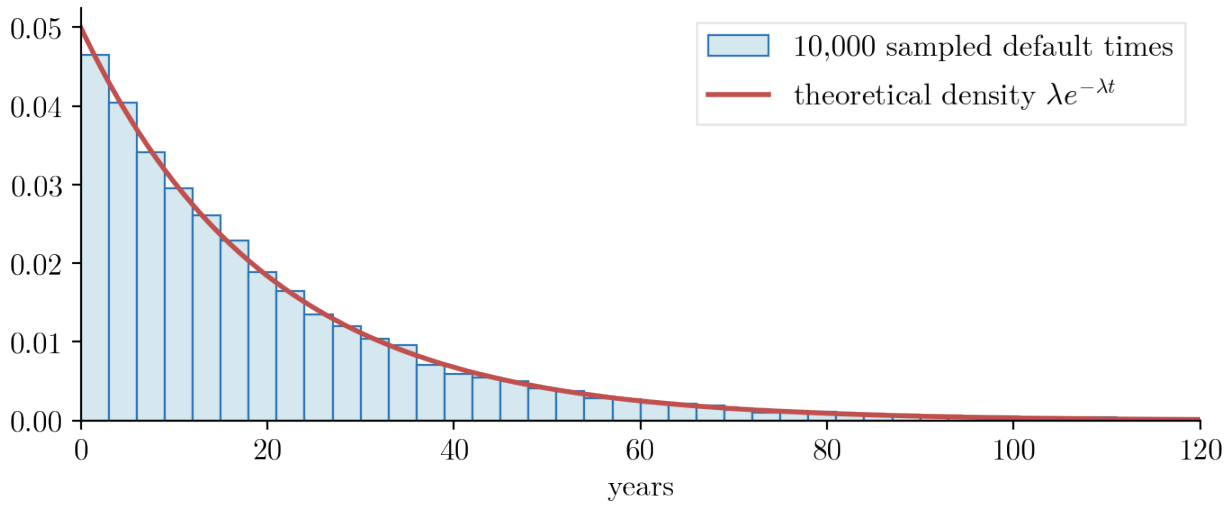


Figure 5.6: *Histogram of 10,000 default times sampled via the inverse-CDF code above ($\lambda = 0.05$), against the theoretical exponential density $\lambda e^{-\lambda t}$.*

This is exactly how SciPy generates exponential random variates internally. NumPy's `np.random.exponential` function does the same thing.

Worked Example: Deriving an Inverse CDF Yourself (Pareto)

The Pareto distribution models fat-tailed quantities such as insurance losses and operational-loss severities. Its CDF, for $x \geq x_m$ (the minimum value) and tail parameter $\alpha > 0$, is $F(x) = 1 - (x_m/x)^\alpha$. To sample from it, set $u = F(x)$ and solve for x: from $u = 1 - (x_m/x)^\alpha$ we get $(x_m/x)^\alpha = 1 - u$, so $x = x_m(1-u)^{-1/\alpha}$. Since $1 - U$ is uniform whenever U is, the sampler is simply $X = x_m \cdot U^{-1/\alpha}$, in NumPy: `x = xm * u**(-1/alpha)`. A quick numerical check with $x_m = 1$ and $\alpha = 2$ shows what a fat tail means in practice: $u = 0.99$ gives $x = (0.01)^{-1/2} = 10$, while $u = 0.9999$ gives $x = 100$. The top 0.01% of draws reaches another factor of ten beyond the top 1%, exactly the kind of behavior a thin-tailed normal cannot produce, and a preview of why tail modeling matters in the credit portfolio applications later in this chapter.

Generating normals from uniforms. The standard normal inverse CDF $\Phi^{-1}(u)$ has no closed form, but SciPy provides `scipy.stats.norm.ppf(u)`. For efficiency, NumPy uses the Ziggurat algorithm directly, but conceptually:

Inverse CDF for normal

```
u = np.random.uniform(0, 1, size=10000)
z = stats.norm.ppf(u) # equivalent to np.random.normal(size=10000)
print(f"Empirical mean: {z.mean():.4f}, std: {z.std():.4f}")
```

Output

```
Empirical mean: 0.0122, std: 1.0009
```

Excel Equivalent

Excel's `RAND()` returns a uniform random number on [0, 1). To get a standard normal, use `=NORM.S.INV(RAND())` for the inverse-CDF method. For an exponential with mean $1/\lambda$, use `=-LN(RAND())/lambda`.

Excel's RNG is acceptable for small simulations but is less robust than NumPy's for serious work, it has period only about 2^{32} and shows correlations at large sample sizes. For more than 100,000 draws, prefer Python.

5.4 The Basic Monte Carlo Loop

Every Monte Carlo simulation follows a fixed five-step pattern. Internalize this pattern and you can write any simulation.

Monte Carlo simulation: the five steps

1. **Model the randomness.** Identify the random variables and their joint distribution.
2. **Generate samples.** Draw N replicates of the random variables from their distribution.
3. **Compute outcomes.** Apply the deterministic mapping to convert each sample into a cash flow (or whatever quantity you care about).
4. **Aggregate.** Compute the sample mean (for expectations), or quantiles (for risk measures), or any other statistic.
5. **Quantify error.** Report a standard error or confidence interval.

In practice, most analysts report both the point estimate (the sample mean) and a MaxErr estimate, typically taken as 2-3 standard errors of the simulation. MaxErr is the practical bound on how far the true value is likely to sit from that estimate, and it is the number a reader actually needs to judge whether the simulation is precise enough to act on.

Worked Example: Valuing a Defaultable Bond by Simulation

In Chapter 4 we valued a defaultable corporate bond analytically using the approximation $YTM = r + \pi\lambda$. Now let's value the same bond by simulation and verify that the answers

agree.

Setup. A 5-year corporate bond, \$100 face, 8% annual coupon, PD = 4% per year (independent each year), LGD = 70%, risk-free rate $r = 4\%$. Each year, given the bond has not yet defaulted, default occurs with probability $\pi = 4\%$. On default, the bondholder receives $(1-\lambda) \times 100 = \$30$ at year-end and no further cash flows. If the bond survives all 5 years, it pays the full schedule.

Step 1: Model the randomness. The random object is the year of default $K \in \{1, 2, 3, 4, 5, \infty\}$, where ∞ denotes survival. Under the independent-default model:

$$P(K = k) = (1-\pi)^{k-1} \cdot \pi \text{ for } k = 1, \ldots, 5$$

$$P(K = \infty) = (1-\pi)^5$$

The tree below illustrates this model. Each year the surviving bond either defaults with probability π (branch down: the path ends, and the bondholder keeps the coupons already collected plus the \$30 recovery) or survives with probability $1-\pi$ (branch right, collecting that year's \$8 coupon). The six terminal nodes are exactly the six values K can take, and every simulated path in the code below is one walk through this tree.

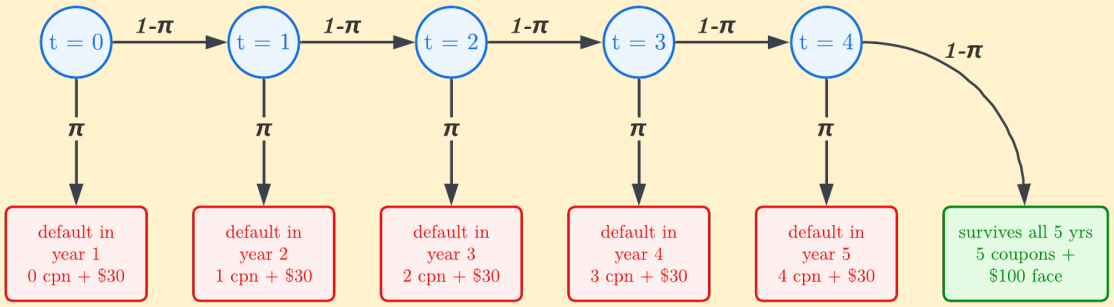


Figure 5.7: *The six mutually exclusive outcomes of the defaultable bond: default in year 1 through 5, or survival to maturity, with the cash flow received on each path.*

Step 2: Generate samples. Draw uniform random numbers for each year, each loan, and check whether default occurs. We simulate $N = 100{,}000$ paths.

Step 3: Compute outcomes. For each path, compute the present value of cash flows:

- If survives all 5 years: PV $= \sum 8/(1.04)^k + 100/(1.04)^5$ for k = 1..5
- If defaults in year k: PV $= \sum 8/(1.04)^j$ *for* $j = 1..k\text{-}1$, *plus* $30/(1.04)^k$

Defaultable bond by Monte Carlo

```
# Defaultable bond valuation by Monte Carlo
np.random.seed(123)
N = 100_000
T = 5
coupon = 8
face = 100
pi = 0.04
LGD = 0.70
r = 0.04

# Step 2: simulate default times
defaults = np.random.uniform(size=(N, T)) < pi # (N, T) Boolean matrix

# Find first default year for each path (0 if survives)
first_default = np.where(
    defaults.any(axis=1),
    defaults.argmax(axis=1) + 1,
    0  # marker for survival
)

# Step 3: compute PV for each path
pvs = np.zeros(N)
discount = (1 + r) ** -np.arange(1, T+1)
for i in range(N):
    d = first_default[i]
    if d == 0:
        # Full schedule: coupons + face
        pvs[i] = coupon * discount.sum() + face * discount[-1]
    else:
        # Coupons until year d-1, then recovery at year d
        if d > 1:
            pvs[i] = coupon * discount[:d-1].sum()
        pvs[i] += (1 - LGD) * face * discount[d-1]

# Step 4: aggregate
price = pvs.mean()
se = pvs.std() / np.sqrt(N)
ci_low = price - 1.96 * se
ci_high = price + 1.96 * se
print(f"Simulated price: ${price:.4f}")
print(f"Standard error: ${se:.4f}")
print(f"95% CI: [${ci_low:.4f}, ${ci_high:.4f}]")
```

Output

```
Simulated price: $103.7343
Standard error: $0.0947
95% CI: [$103.5487, $103.9198]
```

Before summarizing into a single price, look at the whole distribution of the 100,000 path values. It has exactly six bars, one per terminal node of the tree: 81.7% of paths survive and are worth $117.81, and the five default outcomes (worth $28.85 up to $53.70, depending on how many coupons were collected first) each catch 3.4-3.9% of paths. The price $103.73 is just the probability-weighted average of these six numbers; the simulation's real product is the distribution itself.

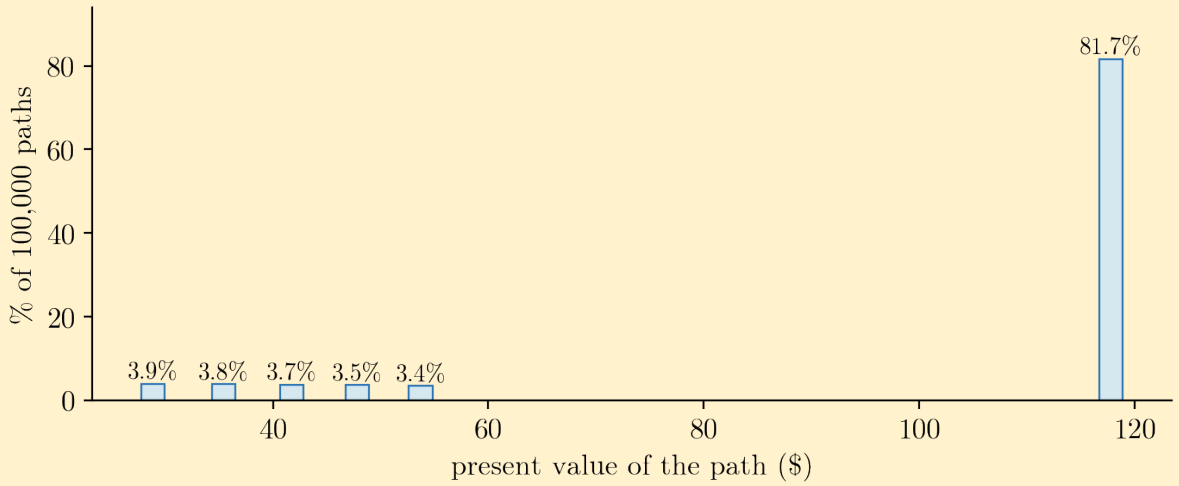


Figure 5.8: *Distribution of present value across the 100,000 simulated bond paths. The six bars correspond to the six terminal nodes of Figure 5.7.*

Step 5: Quantify error. Standard error of $0.09 means we can be 95% confident the true price is within $103.55-$103.92. With 100,000 simulations this is satisfactory for most applications; doubling the sample size would tighten the confidence interval by a factor of $\sqrt{2} \approx 1.41$.

Verification against the analytic price. Using the closed-form formula from Chapter 4:

Analytic verification

```
# Analytic price using year-by-year survival probabilities
exact = 0
for k in range(1, T+1):
    S_prev = (1-pi)**(k-1)
    S_curr = (1-pi)**k
```

```
    prob_default = S_prev - S_curr
    cf_survive = coupon + (face if k == T else 0)
    cf_default = (1-LGD)*face
    exp_cf = S_curr*cf_survive + prob_default*cf_default
    exact += exp_cf / (1+r)**k
print(f"Analytic price: ${exact:.4f}")
```

Output

```
Analytic price: $103.6281
```

Simulation: \$103.73 ± \$0.19. Analytic: \$103.63. Difference: \$0.10, well within the 95% confidence interval. The simulation is correct.

When Does This Matter?

For a bond with constant PD and LGD, simulation is unnecessary, the analytic formula is exact and much faster. Simulation becomes essential when:

- PD or LGD is time-varying or path-dependent;
- the risk-free rate is stochastic;
- multiple bonds default with correlation (the mini-project at the end of the chapter);
- we want the full distribution of outcomes, not just the mean (e.g., for value-at-risk).

The simple example above is a *benchmark*, we used it to verify the simulation works. In real applications the analytic formula is unavailable.

5.5 Variance Reduction

The Monte Carlo error scales as $\sigma/\sqrt{N}$. To halve the error requires $4\times$ the sample size, a steep cost. An attractive alternative is to **reduce** σ while keeping N fixed. This is the goal of variance reduction techniques.

All variance reduction techniques exploit one of three ideas: introduce negative correlation between paired samples (antithetic variates), enforce known moments on the sample (moment matching), or subtract a correlated variable with known mean (control variates). We treat each in turn.

To compare the techniques fairly, this section tests each one on the same small problem: estimate $\mathbb{E}\left[e^Z\right]$ for standard normal Z. The true value is known in closed form, $\mathbb{E}\left[e^Z\right] = e^{0.5} \approx 1.6487$, so we can see exactly how far every estimate lands from the truth. Each technique also gets the same raw material: one fixed set of 50 standard normal draws (`np.random.seed(3)` in the listings below),

so any improvement comes from the technique, not from luckier draws. This particular sample happens to come out low, its plain Monte Carlo average is 1.2571, about 24% below the true value, which makes it a demanding test: a good variance reduction technique should rescue the estimate without a single new draw.

5.5.1 Antithetic Variates

If Z is a standard normal, so is -Z. For each normal draw Z_i, create a paired sample -Z_i and average:

$$\hat{y}_i = (g(Z_i) + g(-Z_i))/2$$

$$\bar{\hat{y}} = \frac{1}{N}\sum \hat{y}_i, \;\; i = 1, \ldots, N/2$$

If g is a monotonic function, $g(Z_i)$ and $g(-Z_i)$ are negatively correlated, and the variance of their average is smaller than the variance of either alone. We get N samples for the cost of N/2 independent draws.

When it works. Antithetic variates work best when the integrand $g(x)$ is approximately linear or monotonic in x. They can fail (or even increase variance) for symmetric integrands like $g(x) = x^2$, where $g(Z) = g(-Z)$.

The intuition: whenever a draw Z_i happens to land above its mean, its mirror image $-Z_i$ lands below by exactly the same amount. Since e^Z rises with Z, one member of the pair pushes the estimate up while the other pulls it down, so the sampling errors partially cancel *within each pair* instead of having to average out slowly across many independent draws. There is also a bookkeeping bonus: 25 random draws produce 50 samples, because each draw is used twice, once with each sign.

Antithetic variates

```
# Estimating E[e^Z]; true value is e^0.5 = 1.6487
np.random.seed(3)
z = np.random.normal(size=50)  # the section's 50 shared draws
true_value = np.exp(0.5)

# Plain MC: average e^z over all 50 draws
plain = np.exp(z).mean()

# Antithetic: the first 25 draws AND their mirror images
# (25 pairs = 50 samples, only 25 random draws consumed)
z25 = z[:25]
pairs = (np.exp(z25) + np.exp(-z25)) / 2 # one average per pair
anti = pairs.mean()

print(f"true value : {true_value:.4f}")
print(f"plain MC : {plain:.4f}  error {plain - true_value:+.4f}")
```

```
print(f"antithetic : {anti:.4f} error {anti - true_value:+.4f}")
print(f"variance ratio: {np.exp(z).var() / (2*pairs.var()):.2f}x")
```

Output

```
true value : 1.6487
plain MC   : 1.2571  error -0.3917
antithetic : 1.5679 error -0.0808
variance ratio: 2.45x
```

From the same underlying draws, pairing cut the error from -0.39 to -0.08, roughly a factor of five, at zero additional sampling cost. Figure 5.9 shows the whole path: the running antithetic estimate settles quickly while the plain running mean is still wandering. The estimate is still a little low because this particular sample is systematically low and mirroring only the first 25 draws cannot undo all of that, but the variance ratio of about 2.5× means 50 antithetic samples deliver the accuracy of roughly 120 plain ones.

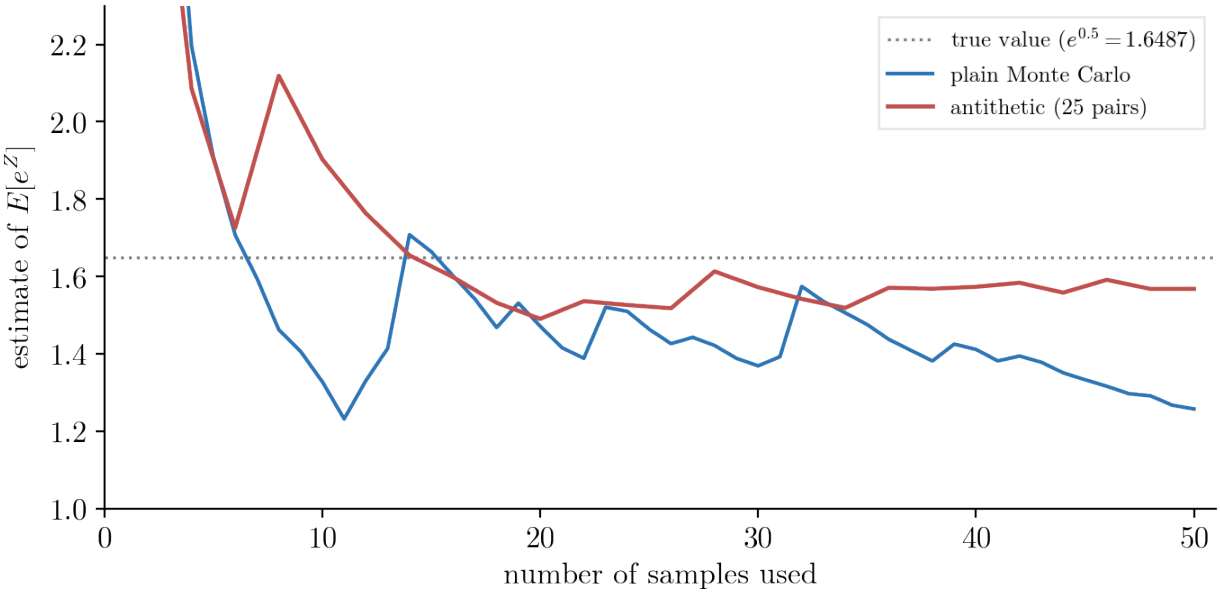


Figure 5.9: *Running estimate of $E[e^Z]$ as samples accumulate, one simulation run (seed 3): plain Monte Carlo over all 50 draws (blue) against antithetic pairs built from the first 25 draws (red). The dotted line is the true value $e^{0.5}$.*

5.5.2 Moment Matching

A sample of N standard normals will not have exactly mean zero and standard deviation one, only approximately so. We can *force* the sample to have these moments by adjusting:

$$\tilde{Z}_i = (Z_i - \bar{Z})/s_Z$$

where $\bar{Z}$ and s_Z are the sample mean and standard deviation. The adjusted sample has exact mean 0 and standard deviation 1.

When it works. Moment matching helps when the function g depends strongly on the first two moments of the input. It always reduces error in expectations of linear functions of Z, and typically helps for nonlinear functions too.

The intuition: the plain estimate was poor for a reason we can actually see. Our 50 shared draws happen to have sample mean -0.29 when the population mean is 0; the whole sample sits shifted to the left. Since e^z is increasing in z, a left-shifted sample must produce an estimate of $\mathbb{E}\left[e^Z\right]$ that is too low. Moment matching repairs the defect at its source: shift and rescale the draws so their mean and standard deviation are *exactly* 0 and 1, then run the same simulation on the corrected sample. We are not adding information; we are removing a sampling accident we know the correct answer to.

Moment matching

```
# Same 50 shared draws (z) and true_value as above
print(f"sample mean = {z.mean():+.4f}, sample sd = {z.std():.4f}")

z_mm = (z - z.mean()) / z.std() # force mean 0, sd 1 exactly
mm = np.exp(z_mm).mean()

print(f"moment matched: {mm:.4f} error {mm - true_value:+.4f}")
```

Output

```
sample mean = -0.2939, sample sd = 0.9985
moment matched: 1.6893 error +0.0406
```

The error drops from -0.39 to $+0.04$, a factor of ten, again with no new draws. Figure 5.10 shows why the improvement arrives so early: once even a modest number of draws is in hand, the rescaled sample has the right center of mass, so the running estimate hugs the true value long before the plain running mean recovers. The small remaining error is the part moment matching cannot fix: two matched moments do not make the sample's whole *shape* normal.

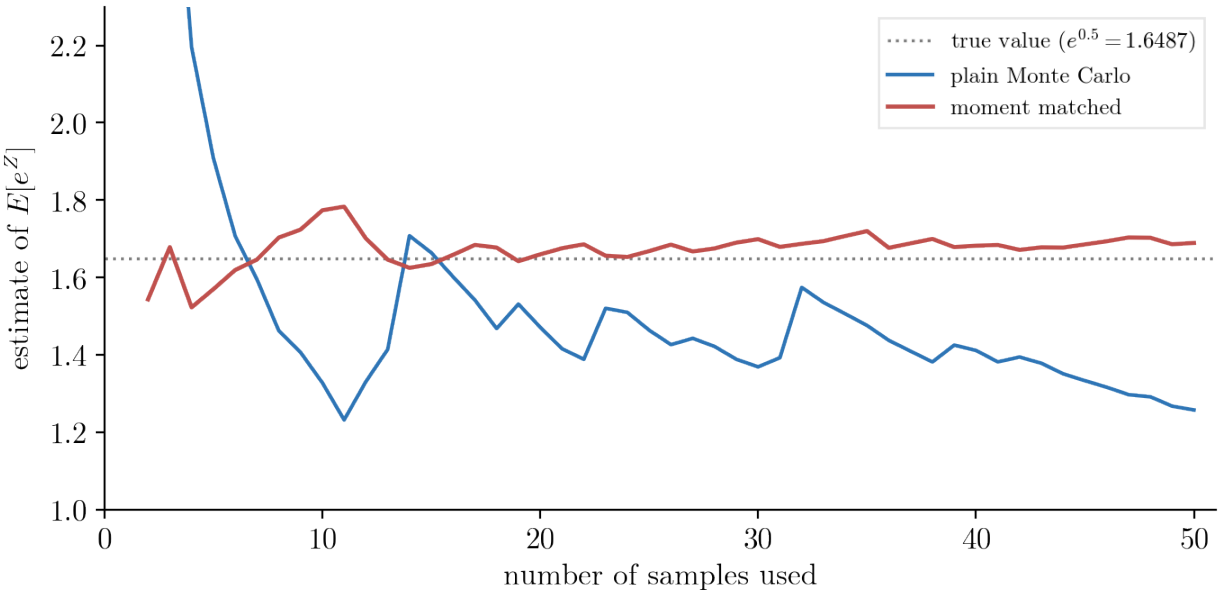


Figure 5.10: *Running estimate of $E[e^Z]$, one simulation run (seed 3): plain Monte Carlo (blue) against the moment-matched estimate (red), where at each sample count n the first n draws are rescaled to exact mean 0 and sd 1.*

5.5.3 Control Variates: One Control

Control variates use a *known mean* of a correlated quantity to reduce variance. Suppose we want to estimate $\mathbb{E}[Y]$, and we know $\mathbb{E}[X] = \mu_X$ exactly. Then the **control variate estimator** is:

$$\hat{Y}_{cv} = \bar{Y} - \beta \cdot (\bar{X} - \mu_X)$$

for some coefficient β. The variance is minimized by setting $\beta^* = \mathrm{Cov}(X, Y) \,/\, \mathrm{Var}(X)$, giving:

$$\mathrm{Var}\left(\hat{Y}_{cv}\right) = \mathrm{Var}(Y) \cdot (1 - \rho^2)$$

where $\rho = \mathrm{Corr}(X, Y)$. The higher the correlation, the greater the variance reduction. With ρ = *0.95*, variance falls 90%.

The intuition: after simulating, we can observe how our particular draws came out in a *second* quantity whose true mean we know exactly. Here the natural choice is Z itself: we know $\mathbb{E}[Z] = 0$, yet our 50 draws have sample mean -0.29. That gap is direct evidence of how unlucky this sample was. And because e^Z co-moves strongly with Z (their sample correlation is 0.86), a shortfall in $\bar{Z}$ translates into a proportional shortfall in the estimate of $\mathbb{E}\left[e^Z\right]$, proportional by exactly the regression slope β. The control variate estimator simply adds back what the shortfall took away.

Control variate (one control)

```
# Same 50 shared draws (z) and true_value as above
y = np.exp(z)
beta = np.cov(z, y)[0, 1] / z.var(ddof=1)
```

```
cv1 = y.mean() - beta * (z.mean() - 0) # E[Z] = 0 is known

print(f"beta = {beta:.4f}, corr(e^Z, Z) = {np.corrcoef(z, y)[0, 1]:.4f}")
print(f"control variate: {cv1:.4f} error {cv1 - true_value:+.4f}")
```

Output

```
beta = 1.3125, corr(e^Z, Z) = 0.8569
control variate: 1.6429 error -0.0059
```

The error falls from -0.39 to -0.006, better than either technique above, and the formula box explains why: with $\rho = 0.86$, the control removes a fraction $\rho^2 \approx 0.73$ of the variance, a factor of nearly four. Figure 5.11 shows the running estimate locking onto the true value once β is pinned down. The strength of a control variate is set entirely by that correlation, which is why finding a good control matters more than any other modeling choice in this section.

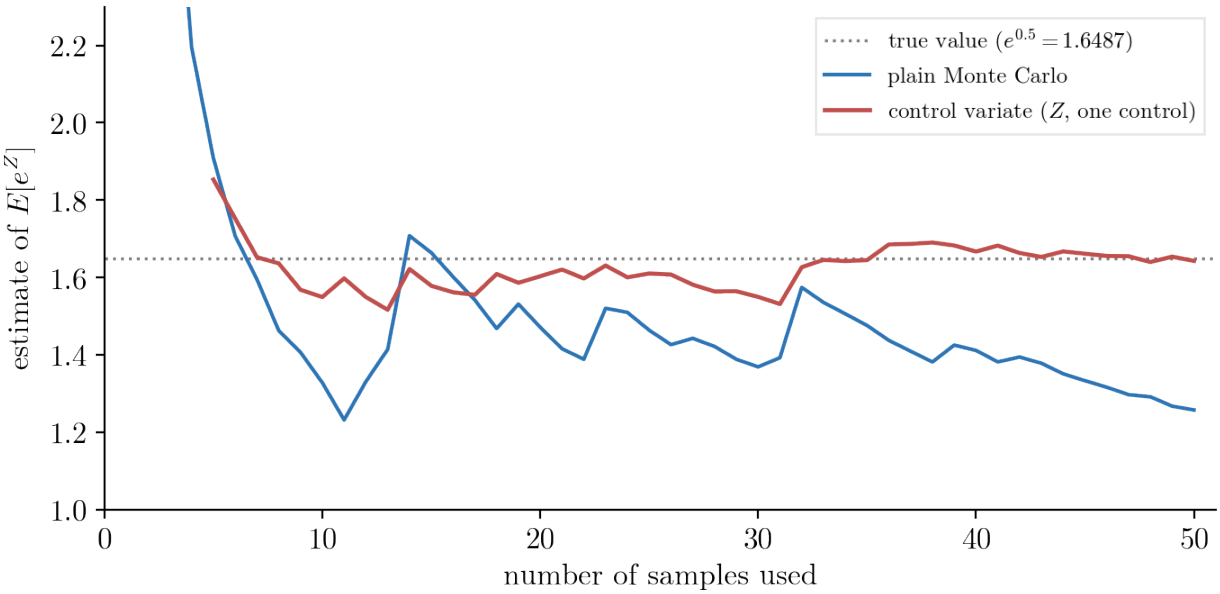


Figure 5.11: *Running estimate of $E[e^Z]$, one simulation run (seed 3): plain Monte Carlo (blue) against the one-control estimator (red), which re-estimates β from the first n draws at each sample count n.*

Figure 5.12 overlays all three techniques on the same run. Every method beats the plain running mean, and they finish in the order the theory predicts: antithetic (-0.08) improves on plain (-0.39), moment matching ($+0.04$) improves on antithetic, and the control variate (-0.006) beats them all. Keep in mind this is a single simulation run: on another seed the finishing order of the middle two can shuffle, but the pattern, techniques that exploit known structure beat raw averaging, is robust. The techniques are also not mutually exclusive: antithetic pairs can be moment matched, and a control variate can be layered on top of either, which is what the last row of the summary table refers to.

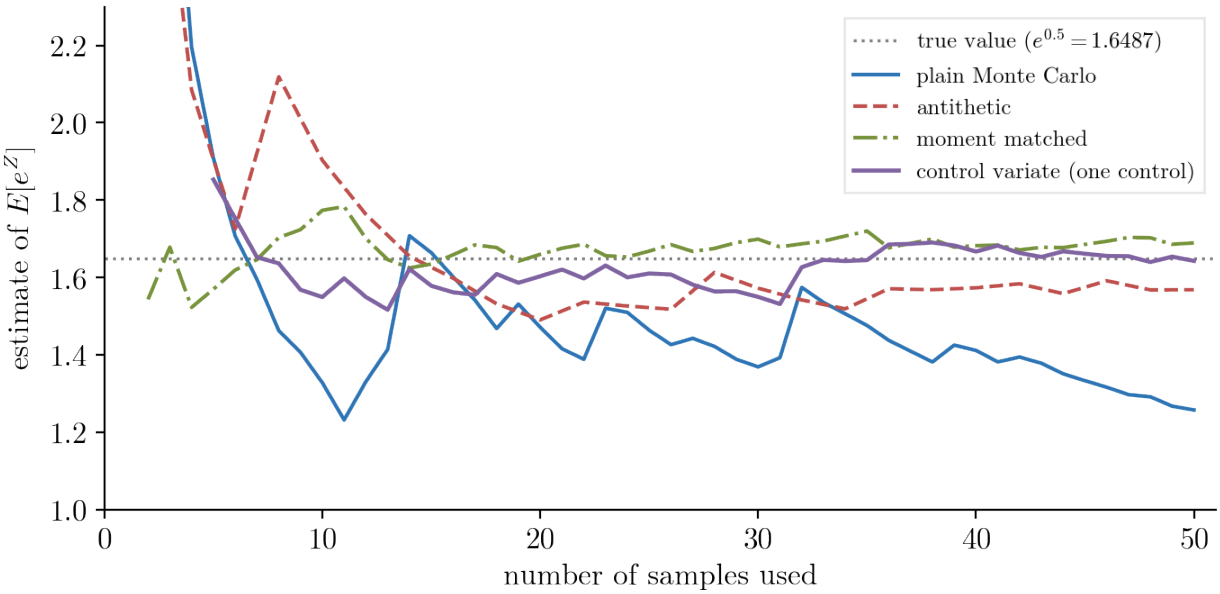


Figure 5.12: *All three variance reduction techniques against plain Monte Carlo on the same 50 draws (seed 3). Each curve is that technique's running estimate of $E[e^Z]$; the dotted line is the true value.*

Variance reduction summary table:

Technique	Best when	Typical reduction
Antithetic	g(x) monotonic in x	1.5-3×
Moment matching	g depends on moments of x	1.1-2×
Control variate	Highly correlated control	5-20×
All three combined	Most problems	10-60×

Variance reductions multiply when techniques are independent. A 60× total reduction means 60× fewer simulations for the same accuracy, or 60× more accurate results with the same compute. On large jobs (e.g., bank-wide credit portfolio simulations) this is the difference between overnight and a coffee break.

5.6 Control Variates with Two Controls

Nothing in the control variate idea limits us to a single control. Any quantity we can compute from the same draws, and whose true mean we know, is a candidate: each one is another known fact our sample can be checked against, and corrected with. For the running benchmark $\mathbb{E}\left[e^Z\right]$ we have been using Z as a control because $\mathbb{E}[Z] = 0$ is known. But we know a second moment of the same draws just as exactly: $\mathbb{E}[Z^2] = 1$, the variance of a standard normal. Using both should let us correct not only for a sample that drifted left or right, but also for one that came out too dispersed or too compressed.

To use two controls at once, it helps to first see the one-control estimator for what it really is: a regression in disguise. Regress the simulated values $y_i = e^{z_i}$ on the control z_i, and evaluate the fitted line at the control's known mean, $z = 0$. The fitted value there is $\bar{y} - \beta(\bar{z} - 0)$, which is *exactly* the control variate estimator of the last section. In words, the regression answers a counterfactual question: *what would our sample average have been, had the sample come out perfectly average in the control?* Seen this way, the generalization is immediate. With two controls, run a multiple regression of y_i on z_i and z_i^2,

$$y_i = b_0 + b_1 z_i + b_2 z_i^2 + e_i$$

and evaluate the fitted function at the controls' known means, $(\mathbb{E}[Z], \mathbb{E}[Z^2]) = (0, 1)$:

$$\hat{Y}_{cv2} = b_0 + b_1 \cdot 0 + b_2 \cdot 1 = b_0 + b_2$$

Each coefficient transfers one known population fact onto our particular sample; the residual e_i is the only part of the variance the controls cannot explain.

Control variates (two controls)

```
# Same 50 shared draws (z), y = e^z, and true_value as above
X2 = np.column_stack([np.ones(50), z, z**2])
b, *_ = np.linalg.lstsq(X2, y, rcond=None) # OLS coefficients
cv2 = b[0] + b[1]*0 + b[2]*1  # fitted value at (E[Z], E[Z^2])

print(f"b0 = {b[0]:.4f}, b1 = {b[1]:.4f}, b2 = {b[2]:.4f}")
print(f"two controls: {cv2:.4f} error {cv2 - true_value:+.4f}")
```

Output

```
b0 = 1.0911, b1 = 1.4964, b2 = 0.5592
two controls: 1.6503 error +0.0016
```

The error shrinks to +0.0016, one part in a thousand, from the same 50 draws that put plain Monte Carlo off by 24%. Figure 5.13 makes the comparison to the previous three techniques directly: the two-control estimate locks onto the true value within roughly a dozen samples and stays there, visibly tighter than antithetic (−0.08), moment matching (+0.04), or the single control (−0.006). The second control earns its keep because e^z is convex: the z^2 term captures curvature the straight-line fit missed, so far less ends up in the residual.

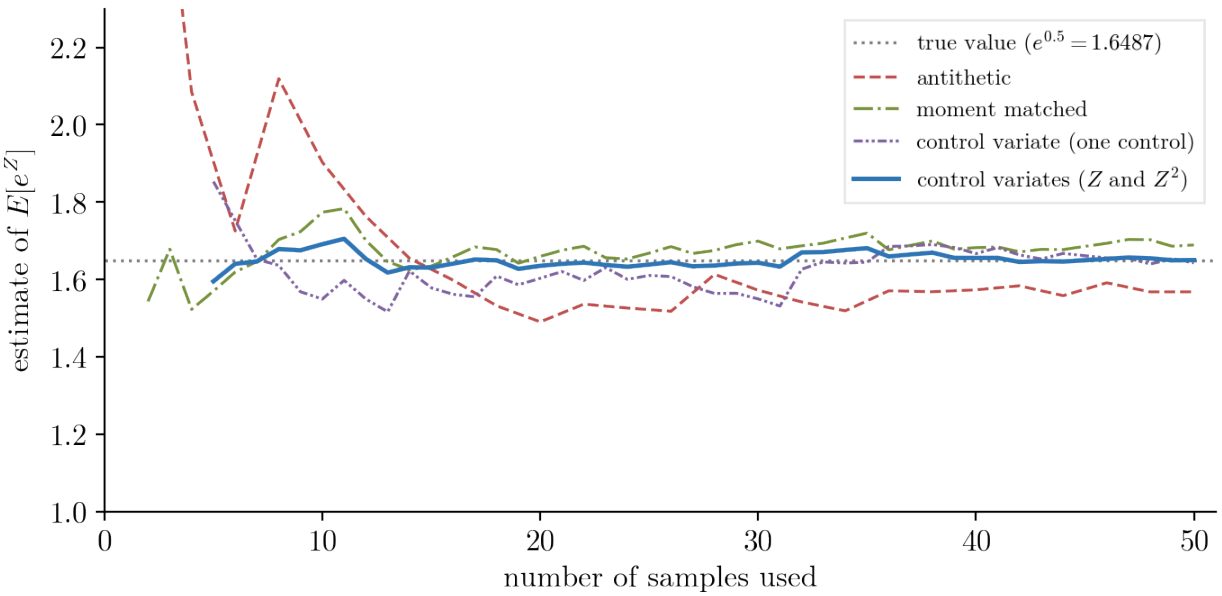


Figure 5.13: *The two-control estimator (Z and Z^2, solid blue) against the three techniques of the previous section, all running on the same 50 draws (seed 3). The regression-based estimate converges fastest and tracks the true value most tightly.*

One warning and one preview. The warning: each added control must bring a genuinely *known* mean. Feed the regression a control whose "known" mean is itself an estimate, and the error of that estimate flows straight into the valuation. The preview: handling two controls forced us to think about how several random quantities co-move, covariances, correlations, joint behavior. That is precisely the machinery the rest of this chapter builds, starting with the problem of simulating correlated random variables directly.

5.7 Multivariate Simulation

Most financial problems involve multiple correlated risks: stock returns within a portfolio, interest rates across the yield curve, credit defaults across an issuer pool. Independent simulation of each risk would miss the correlation that drives most portfolio-level outcomes. We need a method to simulate vectors of correlated random variables, starting with the multivariate normal.

5.7.1 Correlating Two Normals

Simulating *uncorrelated* normals is easy; the problem is correlating them. Start with the smallest version of the problem. We have one standard normal z_1. Can we manufacture a second variable z_2 that (i) has mean 0, (ii) has standard deviation 1, and (iii) has a chosen correlation ρ with z_1? The trick is to blend z_1 with a *fresh, independent* standard normal z_3:

$$z_2 = \rho z_1 + \sqrt{1-\rho^2}\, z_3$$

The two weights are chosen so that the desired properties hold exactly:

- **Mean:** both ingredients have mean 0, so $\mathbb{E}[z_2] = 0$.
- **Variance:** the pieces are independent, so their variances add: $\mathrm{Var}(z_2) = \rho^2 + (1-\rho^2) = 1$.
- **Correlation:** z_3 is independent of z_1, so only the first term co-moves with z_1: $\mathrm{Cov}(z_1, z_2) = \rho\ \mathrm{Var}(z_1) = \rho$.

Read the formula as a recipe with two dials. The first term, ρz_1, copies a fraction of the original variable: it is the *shared* part, the source of all the correlation. The second term, $\sqrt{1-\rho^2}\, z_3$, adds just enough *independent* noise to top the variance back up to 1. The weights are not symmetric (ρ against $\sqrt{1-\rho^2}$) because correlation lives in units of covariance while the top-up lives in units of variance. The two extremes check out: at $\rho = 1$ the noise term vanishes and z_2 is a copy of z_1; at $\rho = 0$ the copy term vanishes and z_2 is simply the independent draw z_3. Following the lecture ritual, we test all three properties on 100,000 simulated pairs:

Correlating two normals

```
np.random.seed(3)
rho = 0.7
z1 = np.random.normal(size=100_000)
z3 = np.random.normal(size=100_000)  # independent of z1
z2 = rho*z1 + np.sqrt(1 - rho**2)*z3

print(f"z2 mean = {z2.mean():+.4f} (target 0)")
print(f"z2 sd  = {z2.std():.4f}  (target 1)")
print(f"corr   = {np.corrcoef(z1, z2)[0, 1]:.4f} (target {rho})")
```

Output

```
z2 mean = -0.0026 (target 0)
z2 sd  = 0.9971   (target 1)
corr   = 0.7006   (target 0.7)
```

Now write the same construction in matrix form. Stacking the recipe for z_1 (which is just "keep z_1") on top of the recipe for z_2:

$$\begin{pmatrix} z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{pmatrix} \begin{pmatrix} z_1 \\ z_3 \end{pmatrix}$$

A lower triangular matrix turns a vector of *independent* normals into a vector of *correlated* ones. Multiply it by its own transpose and something remarkable falls out: the target correlation matrix itself,

$$\begin{pmatrix} 1 & 0 \\ \rho & \sqrt{1-\rho^2} \end{pmatrix} \begin{pmatrix} 1 & \rho \\ 0 & \sqrt{1-\rho^2} \end{pmatrix} = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$$

So the blending weights we derived by hand are exactly a matrix square root of the correlation matrix. That matrix has a name, the *Cholesky factor*, and the next section turns this two-variable trick into the general method: for any number of variables, factor the target covariance matrix and multiply.

5.7.2 The Cholesky Decomposition

Suppose we want to simulate $X = (X_1, \ldots, X_d)$ with mean vector μ and covariance matrix Σ. The Cholesky decomposition factors Σ as:

$$\Sigma = LL^T$$

where L is a lower triangular matrix. Then:

$$X = \mu + L \cdot Z, \textbf{ where } Z = (Z_1, \ldots, Z_d), \textbf{ iid } N(0,1)$$

produces a vector with the desired mean and covariance. The transformation takes independent standard normals (easy to simulate) and converts them into correlated normals (what we need).

Math Review: Why Cholesky Works

Compute the covariance of $X = \mu + LZ$:

$$\text{Cov}(X,\, X) = \text{Cov}(LZ,\, LZ) = L \cdot \text{Cov}(Z,\, Z) \cdot L^T = L \cdot I \cdot L^T = LL^T = \Sigma$$

The mean is $\mathbb{E}[X] = \mu + L \cdot 0 = \mu$ by construction. Since X is a linear combination of normals, it is also multivariate normal. So $X \sim N(\mu, \Sigma)$, as required.

Existence. The Cholesky factorization exists and is unique if and only if Σ is positive definite, i.e., a valid covariance matrix.

Example: three correlated assets.

Cholesky decomposition

```
# Three assets with the following correlation structure
cov = np.array([[1.0, 0.5, 0.3],
                [0.5, 1.0, 0.6],
                [0.3, 0.6, 1.0]])

# Cholesky factorization
L = np.linalg.cholesky(cov)
print("L =")
print(L)
```

```
# Verify: L L^T should equal the covariance
print("\nL L^T =")
print(L @ L.T)
```

Output

```
L =
[[1.   0.   0.  ]
 [0.5  0.866 0.  ]
 [0.3  0.520 0.800]]

L L^T =
[[1.0 0.5 0.3]
 [0.5 1.0 0.6]
 [0.3 0.6 1.0]]
```

Generate correlated samples:

Generating correlated normals

```
np.random.seed(7)
n = 100_000
z = np.random.normal(size=(3, n)) # iid standard normals: 3 x n matrix
x = L @ z # correlated normals: same shape

# Verify empirical covariance matches the target
print("Empirical covariance:")
print(np.cov(x))
```

Output

```
Empirical covariance:
[[0.990 0.493 0.301]
 [0.493 0.999 0.605]
 [0.301 0.605 1.004]]
```

Within sampling noise, the empirical covariance matches the target exactly. With 100,000 samples, the largest discrepancy is less than 1%.

The scatter plots below (first 2,500 simulated points, same seed) put the two most different correlations side by side: the X_2-X_3 cloud (correlation 0.6) is visibly more elongated along the diagonal than the X_1-X_3 cloud (0.3), the difference in dependence strength is immediate on sight, not just in the covariance numbers above.

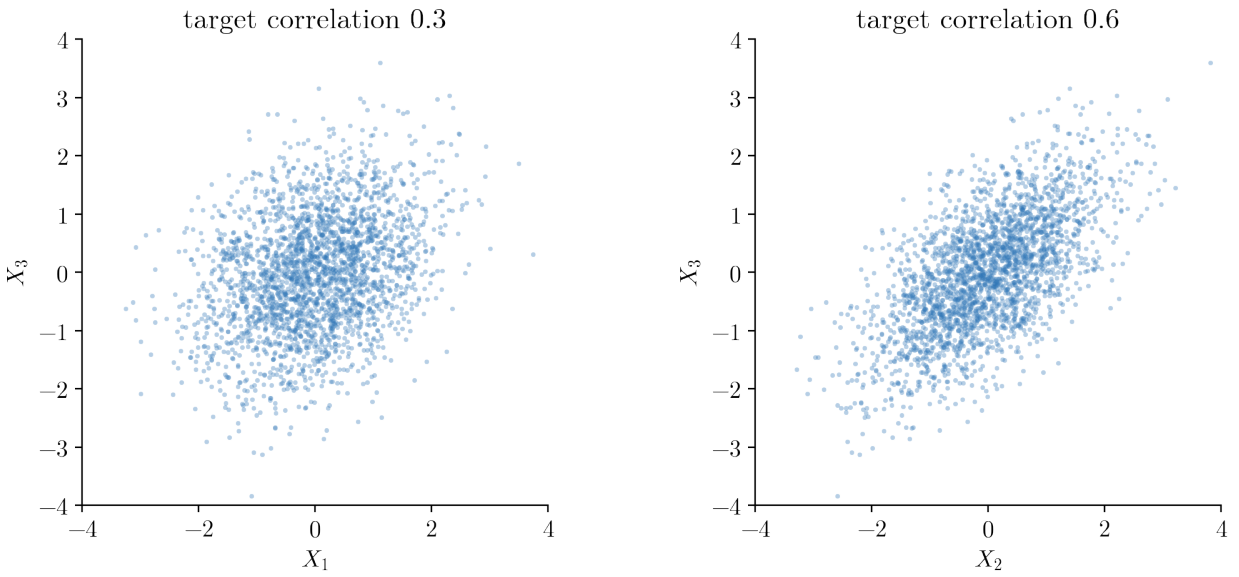


Figure 5.14: *Pairwise scatter plots of 2,500 simulated correlated normal draws, at target correlations 0.3 and 0.6.*

Corporate Application: Portfolio Value-at-Risk by Simulation

A bank's trading book holds positions in three correlated assets. To estimate the one-day 99% Value-at-Risk:

- Estimate the covariance matrix of daily returns from historical data.
- Cholesky-factor the covariance.
- Simulate 100,000 daily return vectors using $r = \mu + L \cdot Z$.
- Compute the portfolio P&L for each scenario: $PnL = w^T r$, where w is the position vector.
- Report the 1st percentile of the PnL distribution as the 99% VaR.

This is the workhorse of bank risk management. Banks run this calculation overnight on portfolios with thousands of positions; correlations are re-estimated daily.

5.8 Non-Normal Dependence: The Gaussian Copula

The Cholesky method works for normal random vectors. But financial variables are rarely jointly normal: stock returns have fat tails, defaults are binary, and many quantities have skewed or bounded distributions. We need a way to simulate correlated variables when the marginal distributions are not normal.

The **Gaussian copula** provides a flexible solution. It decomposes the problem into two parts: the marginal distributions of each variable, and a dependence structure inherited from a multivariate normal. By varying the marginals independently of the dependence, we can build essentially any joint distribution.

5.8.1 The Gaussian Copula Recipe

To simulate $(X_1, \ldots, X_d)$ with marginal CDFs $F_1, \ldots, F_d$ and Gaussian dependence with correlation matrix R:

1. Cholesky-factor $R = LL^T$.

2. Generate iid standard normals $Z = (Z_1, \ldots, Z_d)$.

3. Form correlated normals $Y = L \cdot Z$.

4. Transform to uniforms via the normal CDF: $U_i = \Phi(Y_i)$.

5. Apply each marginal inverse CDF: $X_i = F_i^{-1}(U_i)$.

The result $X = (X_1, \ldots, X_d)$ has the specified marginals and a Gaussian dependence structure.

Why it works. Step 4 transforms normals to uniforms via the probability integral transform. The uniforms U_i are correlated (because the Y_i were). Step 5 applies the inverse CDF for each marginal, producing variables with the right marginal distributions while preserving the rank correlation from the Gaussian step.

Worked Example: Correlated Default Indicators

Suppose two borrowers each have annual default probability $\pi = 5\%$ (marginally Bernoulli with parameter 0.05). We want their defaults to be correlated via a Gaussian copula with correlation $\rho = 0.40$.

Gaussian copula for defaults

```
np.random.seed(2026)
N = 100_000
pi = 0.05
rho = 0.40

# Step 1: Cholesky of 2x2 correlation matrix
R = np.array([[1.0, rho], [rho, 1.0]])
L = np.linalg.cholesky(R)

# Steps 2-3: correlated bivariate normals
z = np.random.normal(size=(2, N))
y = L @ z

# Steps 4-5: transform to defaults via Bernoulli inverse CDF
```

```
# For a Bernoulli(pi), F^-1(u) = 1 if u < pi else 0
# Equivalently, default = 1 if Y < Phi^-1(pi)
default_threshold = norm.ppf(pi)
defaults = (y < default_threshold).astype(int)

# Empirical marginal default rates and correlation
print(f"Marginal default rate 1: {defaults[0].mean():.4f} (target: {pi})")
print(f"Marginal default rate 2: {defaults[1].mean():.4f} (target: {pi})")
print(f"Empirical correlation: {np.corrcoef(defaults)[0,1]:.4f}")
print(f"Joint default rate: {(defaults[0] & defaults[1]).mean():.4f}")
print(f"Independent baseline: {pi**2:.4f}")
```

Output

```
Marginal default rate 1: 0.0496 (target: 0.05)
Marginal default rate 2: 0.0501 (target: 0.05)
Empirical correlation: 0.1762
Joint default rate: 0.0093
Independent baseline: 0.0025
```

The marginal default rates exactly match the target 5%. The joint default rate (0.93%) is *3.7× the independent baseline* of 0.25%. This is the practical impact of correlation: it disproportionately raises the probability of joint extreme outcomes, the source of "tail risk" in credit portfolios.

Two things are worth unpacking in these numbers. First, restate the joint rate as a *conditional* probability: given that borrower 1 has defaulted, the probability that borrower 2 also defaults is the joint rate divided by the marginal rate, $0.0093/0.0496 \approx 19\%$. Learning that one borrower failed nearly quadruples our assessment of the other's risk, from 5% to 19%, even though nothing about borrower 2 itself has changed. A loan officer who prices each borrower in isolation at 5% is correct one borrower at a time and badly wrong about the portfolio. Second, the mechanism: in step 3 the correlated normals are built as $Y = L \cdot Z$, so both borrowers' latent variables share a common component of Z. A bad draw of that shared component, think of it as the state of the business cycle, drags *both* latent variables toward the default threshold at once. Correlation does not change how often each borrower defaults (the marginals are pinned at 5% by construction); it changes *when* they default, bunching failures into the same scenarios. That bunching is invisible in expected values and shows up exactly where it hurts: the tail.

A Note of Caution

The Gaussian copula was famously implicated in the 2008 financial crisis. CDO traders used it to price portfolios of mortgage-backed securities, but the copula's thin-tailed Gaussian depen-

dence systematically *underestimated* the probability of simultaneous defaults in extreme stress scenarios. Real correlations rise sharply in crises ("contagion"); the Gaussian copula assumes correlations are stable. The technique remains useful as a workhorse for routine risk modeling, but quantitative risk managers now supplement it with stress tests and alternative copulas (t-copula, Clayton, Gumbel) that capture tail dependence more accurately.

The precise property at fault has a name: the Gaussian copula has *zero tail dependence*. Take the conditional probability from the worked example, the chance borrower 2 defaults given borrower 1 has, and push the default threshold further and further into the tail (rarer and rarer default events). Under a Gaussian copula that conditional probability drifts toward *zero* for any correlation below 1, so the model quietly asserts that truly extreme failures happen alone. Crisis experience says the opposite: the worse the event, the more likely it drags others with it. Copulas in the t and Clayton families keep that conditional probability positive no matter how extreme the event, which is what "capturing tail dependence" means.

A short tour of copula families. For the reader who wants to go further, here is a map of the main copula families and the jobs they are hired for. Each name below is the exact term to search.

Sklar's theorem is the license for the whole enterprise: any joint distribution can be split into its marginal distributions plus a copula that carries all of the dependence, and conversely any copula can be glued to any marginals to make a valid joint distribution. This is why we could model 5% Bernoulli defaults with normal dependence above: marginals and dependence are separate dials.

The Gaussian copula is the one built in this section: dependence inherited from a multivariate normal with correlation matrix R. Symmetric (upside and downside dependence are equal), a single parameter per pair, fast to simulate, and no tail dependence. It is the default workhorse for routine risk aggregation, and the wrong tool when joint extremes are the whole question.

The Student-t copula is the Gaussian's fat-tailed sibling: dependence inherited from a multivariate t distribution. One extra parameter, the degrees of freedom, controls tail dependence in *both* tails at once; small values mean extremes cluster strongly, and as the degrees of freedom grow, the t copula converges back to the Gaussian. It is the standard first upgrade for equity portfolios and stress testing, where crashes and melt-ups both cluster.

The Clayton copula has *lower*-tail dependence only: joint crashes are common, joint booms are not. That asymmetry matches credit and housing portfolios, where borrowers fail together in recessions but do not prosper together in booms, which is why Clayton is the usual exhibit for what the Gaussian copula missed in 2008. It belongs to the *Archimedean* family, copulas built from a one-parameter generator function rather than a correlation matrix, which makes it easy to write down but harder to scale to many dimensions.

The Gumbel copula, also Archimedean, is Clayton's mirror image: *upper*-tail dependence only. It suits insurance and operational-loss aggregation, where the question is whether the largest losses arrive together (a hurricane season, a systems failure cascading across business lines). Between them, Clayton and Gumbel let a modeler choose *which* tail clusters; the t copula forces both;

the Gaussian allows neither.

5.9 Common Random Numbers for Sensitivity and Optimization

Suppose we want to compare the value of a financial structure under two different parameter sets. For example: how does a corporate bond's value change if PD rises from 4% to 5%?

Naively, we would run two independent Monte Carlo simulations. But this introduces noise into the comparison: each simulation has its own sampling error, and the *difference* between the two values inherits both. With $\sigma \approx$ *$0.10* standard error on each run, the standard error on the difference is $\sqrt{2} \times 0.10 \approx \$0.14$, potentially larger than the sensitivity itself.

The fix: use common random numbers (CRN). Run both simulations using *the same* random draws, same uniforms, same Cholesky-factored normals, same default times. Differences across parameter sets reflect *only* the parameter change, not sampling noise.

Common random numbers

```
# Sensitivity of bond price to PD using common random numbers
np.random.seed(42) # fixed seed for both runs
N = 100_000
T = 5

# Generate uniforms ONCE
u = np.random.uniform(size=(N, T))

def price_bond(u_random, pi, LGD=0.70, r=0.04, coupon=8, face=100):
    defaults = u_random < pi
    first_default = np.where(
        defaults.any(axis=1),
        defaults.argmax(axis=1) + 1, 0)
    discount = (1 + r) ** -np.arange(1, T+1)
    pvs = np.zeros(len(u_random))
    for i in range(len(u_random)):
        d = first_default[i]
        if d == 0:
            pvs[i] = coupon*discount.sum() + face*discount[-1]
        else:
            if d > 1:
                pvs[i] = coupon*discount[:d-1].sum()
            pvs[i] += (1-LGD)*face*discount[d-1]
    return pvs.mean(), pvs.std()/np.sqrt(len(u_random))

# Run two scenarios with the SAME random numbers
base_price, base_se = price_bond(u, pi=0.04)
stressed_price, stressed_se = price_bond(u, pi=0.05)
```

```
print(f"Base (PD=4%): ${base_price:.4f} (SE ${base_se:.4f})")
print(f"Stressed (PD=5%): ${stressed_price:.4f} (SE ${stressed_se:.4f})")
print(f"Sensitivity: ${stressed_price - base_price:.4f}")
```

Output

```
Base (PD=4%): $103.7390 (SE $0.0947)
Stressed (PD=5%): $100.5295 (SE $0.1024)
Sensitivity: $-3.2096
```

The full percentage-point stress moves the price by about $3.21, large enough to survive the ±$0.14 noise of two independent runs. But numerical derivatives are computed from much smaller perturbations: bump PD by a tenth of a point instead and the signal shrinks to roughly $0.32 while independent-run noise stays at ±$0.14, nearly half the size of the quantity being measured. With CRN, the noise cancels almost entirely. CRN is essential for any application requiring derivatives, sensitivities, or optimization: bank capital allocation, portfolio rebalancing, scenario analysis.

5.10 Choosing a Method: When to Simulate

With this toolkit in hand, when should a financial engineer reach for Monte Carlo simulation versus the alternatives? Five rules of thumb:

Method	Best when	Worst when
Closed-form formula	Available (Black-Scholes, ZCB)	Model deviates from textbook
Numerical integration	1-2 dimensions, smooth integrand	High dimensions (curse)
Binomial/trinomial tree	Path-dependent, exotic options	Very high dimensions or many factors
PDE methods	Continuous-time models, 1-2 factors	More than 3 state variables
Monte Carlo simulation	Anything else, esp. high-dim, path-dep	Slow convergence at low dimension

The big picture. Monte Carlo trades accuracy for flexibility. For problems where flexibility is essential, correlated credit portfolios, path-dependent derivatives, multi-factor risk models, simulation is the only viable approach. For problems where a closed-form or low-dimensional numerical method works, those alternatives should be used. The key skill is recognizing which problem is in front of you.

The road ahead. In Chapter 6 we introduce stochastic processes for financial variables, the Arithmetic Random Walk, Geometric Random Walk, and Ornstein-Uhlenbeck (mean-reverting)

processes. Simulation is the natural tool for these models. Together, this chapter and the next form the simulation toolkit that we use throughout the second half of the book.

Appendix 5.A: Variance of the Mean Response

Optional material. This appendix develops the statistical theory of the Monte Carlo estimator. Instructors who teach simulation visually in class may treat this material as optional self-study; the main chapter is self-contained without it. Students who want a deeper understanding of standard errors, confidence intervals, and sample sizing should work through this material carefully.

A.1 Estimating the Standard Error

The chapter established that the Monte Carlo estimator $\hat{\mu}_N = \frac{1}{N}\sum_{i=1}^{N} g(X_i)$ is unbiased and carries a standard error of $SE(\hat{\mu}_N) = \sigma/\sqrt{N}$, where $\sigma^2 = \mathrm{Var}(g(X))$ is the variance of a single simulated outcome. This appendix builds the practical machinery on top of that fact.

In practice σ is unknown, we estimate it from the same sample we use to estimate μ. The natural estimator is the sample standard deviation:

$$s^2 = \frac{1}{N-1}\sum(g(X_i) - \hat{\mu}_N)^2$$

The **estimated standard error** is then:

$$\widehat{SE}(\hat{\mu}_N) = s/\sqrt{N}$$

In NumPy, the standard error of a Monte Carlo estimate is:

Standard error in NumPy

```
# X: array of N simulated outcomes
mc_estimate = X.mean()
mc_se = X.std(ddof=1) / np.sqrt(len(X)) # ddof=1 for sample SD
```

(The `ddof=1` argument applies Bessel's correction, dividing by *N-1* instead of *N*, a small adjustment that doesn't matter for *N > 100*.)

A.2 Confidence Intervals

By the Central Limit Theorem, the Monte Carlo estimator is approximately normal for large N:

$$\hat{\mu}_N \approx N(\mu, \sigma^2/N)$$

This gives us a standard tool for reporting uncertainty: the **(1-α) confidence interval** is:

$$\hat{\mu}_N \pm z_{\alpha/2} \cdot s/\sqrt{N}$$

Common choices: 95% confidence uses $z = 1.96$; 99% confidence uses $z = 2.576$; 90% confidence uses $z = 1.645$.

Interpretation. A 95% confidence interval means: if we repeated the entire Monte Carlo experiment many times, 95% of the resulting intervals would contain the true value μ. It does *not* mean there is a 95% probability the true value lies in this particular interval, the true value is fixed, the interval is random.

Example from §5.4. The defaultable bond simulation gave:

- Mean: \$103.7343
- Standard error: \$0.0947
- 95% CI: \$103.55 to \$103.92 (using $103.73 \pm 1.96 \times 0.0947$)

The analytic price (\$103.63) lies inside the 95% CI. As expected.

A.3 Sample Size Determination

How many Monte Carlo paths N do we need? Suppose we want the standard error of the estimate to be at most some target ε:

$$\sigma/\sqrt{N} \leq \varepsilon \Rightarrow N \geq \sigma^2/\varepsilon^2$$

In practice we estimate σ from a pilot run (e.g., $N = 1000$ paths) and use that estimate to plan the full run:

Sample size planning

```
# Pilot run
N_pilot = 1000
pilot_payoffs = simulate(N_pilot) # whatever the simulation does
sigma_hat = pilot_payoffs.std(ddof=1)

# Plan full run for SE = 0.001 (e.g., 0.1 cent on a bond price)
target_se = 0.001
N_full = int(np.ceil((sigma_hat / target_se)**2))
print(f"Pilot std: {sigma_hat:.4f}")
print(f"Required N for SE <= {target_se}: {N_full:,}")
```

Common pitfall: target the wrong precision. For a bond priced at \$100, a standard error of \$0.10 is usually plenty, less than the bid-ask spread. Targeting \$0.001 might require 10,000× more compute for no practical benefit. Match the target precision to the application.

The spreadsheet below makes the $\sigma/\sqrt{N}$ scaling law tangible rather than abstract, using Prof. Shimko's oil-payday class example: a payday is production (normal, mean 1,500, sd 500) times a price factor (exponential, mean 50, sampled via the same inverse-CDF trick as §5.3). The left column holds one run of 1,000 simulated paydays; its standard deviation is about 78,372. Excel's Data Table feature (right, green column) then reruns that entire 1,000-draw experiment 1,000 more times and records each run's average, a simulation of the sampling distribution of the sample mean itself. The standard deviation of those 1,000 averages is about 2,579, and the ratio of the two standard deviations, 30.39, closely matches $\sqrt{1000} \approx 31.6$: squaring that ratio (924) closely recovers $N = 1000$, confirming empirically that $\mathrm{Var}(\bar{X}) = \sigma^2/N$.

	one run: 1,000 paydays	1,000 run averages
1	107,927	75,309
2	29,850	71,961
3	343,569	74,221
4	75,881	76,849
5	128,230	76,263
6	89,284	73,795
7	135,391	77,955
8	14,055	73,739
⋮	⋮	⋮
Average	75,309	75,029
StDev	78,372	2,579
StErr	2,478	82
MaxErr	4,957	163

A.4 Limitations of the Normal Approximation

The CLT-based confidence interval assumes the sample mean is approximately normal. This works well when:

- The sample size is large (typically $N \geq 1000$ suffices for most problems);
- The integrand $g(X)$ has finite variance;
- The distribution of $g(X)$ is not too skewed or fat-tailed.

When the normal approximation fails. Some financial applications produce extremely skewed or fat-tailed payoffs:

- Deep out-of-the-money option pricing: most paths produce zero payoff; a few produce large positive payoffs. The sample mean is dominated by rare events.
- Insurance losses with catastrophic tail: occasional huge losses dominate.

- Credit portfolios with heavy concentration risk.

In these cases the CLT-based CI may be unreliable. Robust alternatives include the bootstrap, jackknife, or stratified sampling. We do not pursue these here, but flag them as topics for further study in advanced simulation courses.

5.11 Key Terms

Term	Definition
Antithetic variates	Variance reduction technique using paired negatively-correlated samples (Z, -Z).
Cholesky decomposition	Factoring a positive-definite matrix Σ as LL^T, used to generate correlated normals.
Common random numbers	Using the same random draws across multiple simulations to reduce noise in sensitivity analysis.
Control variate	Variance reduction technique that subtracts a correlated quantity with known mean.
Curse of dimensionality	Exponential growth of grid points in numerical integration as dimension increases.
Gaussian copula	A method for combining marginal distributions with a Gaussian dependence structure.
Inverse-CDF method	Sampling from any distribution via $X = F^{-1}(U)$, where U is uniform.
Monte Carlo estimator	The sample mean $(1/N)\sum g(X_i)$ as an estimate of $\mathbb{E}[g(X)]$.
Moment matching	Variance reduction by rescaling samples to have exact target moments.
Pseudorandom number	Deterministic sequence designed to pass statistical tests for randomness.
Standard error	Standard deviation of an estimator: $\sigma/\sqrt{N}$ for Monte Carlo.
Variance reduction	Techniques to reduce simulation error without increasing N.

5.12 Interview Questions

Answer each in 2-3 sentences. Show familiarity with terminology and the conceptual structure of simulation.

1. Why is Monte Carlo simulation called "Monte Carlo integration"? Explain the connection.
2. Why does Monte Carlo simulation become more attractive (relative to deterministic integration) as the number of random variables grows?
3. What is the inverse-CDF method, and why does it work?
4. Describe the five steps of a Monte Carlo simulation in order.
5. Why does Monte Carlo error scale as $1/\sqrt{N}$? What does this imply about the cost of high precision?
6. Explain antithetic variates. When do they help most? When can they hurt?
7. What is the Cholesky decomposition, and how is it used to generate correlated normal random vectors?
8. Why might one use a Gaussian copula instead of just a multivariate normal? Give an example.
9. What is the rationale for using common random numbers in sensitivity analysis?
10. Suppose your Monte Carlo standard error is too large. List three ways to reduce it.
11. What is the difference between bias and standard error in a simulation estimator? Which does N affect?
12. When would you choose closed-form valuation over Monte Carlo? When the reverse?

5.13 Numerical and Python Problems

Difficulty: S = Straightforward (mechanical), M = Moderate (combining ideas), C = Challenge (multi-step or open-ended).

1. [S] Inverse-CDF for a Triangular Distribution. A triangular distribution on [a, b] with peak at c has CDF:

$$F(x) = (x-a)^2/[(b-a)(c-a)] \text{ for } a \le x \le c$$

$$F(x) = 1-(b-x)^2/[(b-a)(b-c)] \text{ for } c \le x \le b$$

Derive the inverse CDF $F^{-1}(u)$, then write Python code to generate 10,000 triangular(0, 1, 2) samples and verify the empirical mean equals 1.

2. [S] Simpson vs. Monte Carlo in 1D. Compute $\int_0^1 x^2 e^{-x}\,\mathrm{d}x$ two ways: (a) Simpson's rule with n = 11, 51, 101 grid points; (b) Monte Carlo with N = 100, 1000, 10000 samples drawn uniformly from [0, 1]. Compute the error for each method (the true value is approximately 0.1606). Plot error vs. number of evaluations on a log-log scale.

3. [S] Basic Monte Carlo Loop. Price a European call option with $S_0 = 100$, $X = 100$, $\sigma = 0.25$, $T = 0.5$, $r = 0.03$. (a) Compute the analytic Black-Scholes price. (b) Simulate $N = 10000$ terminal stock prices and price the option by Monte Carlo. Report the simulated price, standard error, and 95% confidence interval. (c) Verify the analytic price falls inside the CI.

4. [M] Antithetic Variates. For the call option in Problem 3, implement antithetic variates with N/2 pairs (total N = 10000 evaluations). Report the variance reduction ratio compared to plain Monte Carlo. Compare it to the roughly 2.5× ratio achieved for $\mathbb{E}\left[e^Z\right]$ in §5.5, and explain why monotonicity of the payoff in Z is what makes the pairing effective in both cases. What kind of payoff would make antithetic variates useless, or worse?

5. [M] Inverse-CDF for Default Times. A bond has constant hazard rate $\lambda = 0.05$ per year, so the default time is exponentially distributed with rate 0.05. (a) Sample $N = 10000$ default times. (b) Plot the empirical CDF. (c) Compute the empirical 5-year survival probability and compare to $e^{-0.25} = 0.7788$.

6. [M] Cholesky for Three-Asset Portfolio. Three stocks have annual expected returns μ = *(10%, 12%, 8%)*, volatilities σ = *(20%, 25%, 15%)*, and correlation matrix:

$$R = [[1.0,\ 0.5,\ 0.3],\ [0.5,\ 1.0,\ 0.4],\ [0.3,\ 0.4,\ 1.0]]$$

(a) Construct the covariance matrix Σ. (b) Cholesky-factor it. (c) Simulate 50,000 annual return vectors. (d) For an equally-weighted portfolio, compute mean return, volatility, and 5% Value-at-Risk. (e) Repeat the construction for *five* standard normal variables with a common pairwise correlation of 0.2, simulate 100 instances, and verify the empirical correlation matrix against the target.

7. [M] Gaussian Copula for Two Correlated Defaults. Two issuers each have annual default probability 5%. Use a Gaussian copula with correlation 0.50 to simulate 50,000 paired default outcomes over one year. Compute (a) the empirical marginal default rate for each issuer, (b) the empirical joint default rate, (c) the conditional default rate of issuer 2 given issuer 1 has defaulted. Compare the joint rate to the independent baseline ($5\% \times 5\% = 0.25\%$).

8. [M] Common Random Numbers for Sensitivity. Take the defaultable bond from §5.4 (5y, 8% coupon, PD = 4%, LGD = 70%, r = 4%). Using common random numbers, estimate $\partial V/\partial\pi$: the sensitivity of bond value to a 1 percentage point increase in PD. Report the sensitivity with and without CRN, and compare the standard errors of each estimate.

9. [C] Control Variate. Reprice the defaultable bond of §5.4 (5y, \$100 face, 8% coupon, PD = 4%/year, LGD = 70%, r = 4%) by Monte Carlo with $N = 10000$ paths, using the survival indicator (1 if the bond never defaults, 0 otherwise) as a control variate. Its true mean is known exactly: $P(\textit{survival}) = 0.96^5 \approx 0.8154$. (a) Estimate β from the simulated paths and compute the CV-adjusted price. (b) Compare the standard errors of the plain and CV-adjusted estimates, and report the variance reduction ratio. (c) Explain why this control is so effective by computing the correlation between the bond's PV and the survival indicator. (d) Propose and test a *second* control with a known mean (for example, the indicator of default in year 1), in the spirit of §5.6.

10. [C] Variance Decomposition. For a portfolio of n iid default indicators, each with probability π, show analytically that the variance of the number of defaults is $n\pi(1\text{-}\pi)$. Now repeat with a Gaussian copula with pairwise correlation ρ. Use simulation to plot the standard deviation of total defaults as a function of ρ (for $n = 100$, $\pi = 5\%$, ρ from 0 to 0.5). Discuss the implications for credit portfolio risk.

Mini-Project 5.1: Correlated Default Risk in an Energy Loan Portfolio

Background. This project applies every technique in the chapter to a realistic credit portfolio problem. The setup mirrors the kind of analysis that bank risk managers perform routinely, and that the 2008 financial crisis exposed as inadequate when correlations were underestimated.

Setup. A regional bank holds 50 corporate loans to companies in the U.S. energy sector. Each loan has:

- Face value: \$2,000,000
- Annual coupon: 6.0%
- Maturity: 5 years
- Annual default probability: PD = 5% per year (independent across years for a given borrower)
- Loss given default: LGD = 60% conditional on default
- Risk-free rate for discounting: r = 4%

Because all borrowers operate in the same sector, their defaults are correlated. Model dependence using a Gaussian copula with pairwise asset correlation $\rho = 0.30$ (a typical assumption for same-sector exposures).

Tasks.

- **1.** Compute the analytic expected portfolio value assuming *independent* defaults. This is your sanity check against simulation.
- **2.** Build a Monte Carlo simulation for the *independent* case. Run $N = 50{,}000$ portfolio paths. Verify the simulated mean matches the analytic mean. Report: mean, standard deviation, and 1% and 5% Value-at-Risk (the 1st and 5th percentiles of the portfolio value distribution).
- **3.** Now introduce correlation. Implement the Gaussian copula default model: in each year, for each surviving loan, default occurs if a latent asset return falls below the threshold $\Phi^{-1}(\pi)$. The latent returns are correlated across borrowers with correlation $\rho = 0.30$.
- **4.** Re-run with correlation. Report the same statistics. Compare to the independent case.

- **5.** Sensitivity to ρ. Run the simulation for $\rho \in \{0, 0.1, 0.2, 0.3, 0.5\}$. Plot the standard deviation of portfolio value and the 1% VaR against ρ. What do you observe?
- **6.** Write a one-page memo to the bank's Chief Risk Officer summarizing the key findings.

Expected results (approximate).

- Analytic mean (independent): $\approx$ \$94.7M
- Simulated mean (independent or correlated, both): $\approx$ \$94.7M (mean is unchanged by correlation)
- Std deviation (independent): $\approx$ \$3.8M
- Std deviation (correlated, $\rho = 0.30$): $\approx$ \$8.7M
- 1% VaR (independent): $\approx$ \$85.7M
- 1% VaR (correlated, $\rho = 0.30$): $\approx$ \$69.3M

Key insight to draw out. Correlation has **essentially no effect on the mean** but **dramatically increases tail risk.** The 1% VaR, the worst case the bank should plan for with 99% confidence, drops from \$85.7M to \$69.3M when correlation rises from 0 to 0.30. That is **\$16M** of additional potential loss in a tail scenario, with no change in expected outcome. This is exactly why correlated default modeling is essential: portfolio expected values can look identical to a regulator while the actual risks differ by a factor of two or more.

Deliverable. Submit a Jupyter notebook containing all code, plots, and the one-page memo. The memo should explain to the CRO (non-technical audience) why a model that ignores correlation would systematically understate portfolio risk, and what specific actions the bank should consider (e.g., diversification limits, sector concentration limits, capital buffers).

Stretch goals. (Advanced students.) (a) Replace the Gaussian copula with a t-copula (heavier tails) and compare risk metrics. (b) Allow LGD to be stochastic (e.g., Beta distribution with mean 0.60) and re-run the simulation. (c) Add a stress test: compute portfolio value if PD doubles to 10% over years 3-5 (a simulated recession). How much does this change the VaR?

5.14 References and Further Reading

Glasserman, Paul (2003). *Monte Carlo Methods in Financial Engineering.* Springer. The definitive reference for financial Monte Carlo, with comprehensive treatment of variance reduction and dependence modeling.

McLeish, Don L. (2011). *Monte Carlo Simulation and Finance.* Wiley. A solid undergraduate treatment with strong examples.

Brandimarte, Paolo (2014). *Handbook in Monte Carlo Simulation: Applications in Financial Engineering, Risk Management, and Economics*. Wiley. Practitioner-focused with extensive examples.

Salmon, Felix (2009). "Recipe for Disaster: The Formula That Killed Wall Street." *Wired*, February 23, 2009. A widely-read journalistic account of the Gaussian copula's role in the 2008 financial crisis.

Sklar, Abe (1959). "Fonctions de répartition à n dimensions et leurs marges." *Publications de l'Institut de Statistique de l'Université de Paris* 8: 229-231. The original copula paper, establishing that any multivariate distribution can be decomposed into marginals and a copula.

Online resources:

- NumPy random documentation: `numpy.org/doc/stable/reference/random/`
- SciPy stats documentation: `docs.scipy.org/doc/scipy/reference/stats.html`
- MIT OpenCourseWare 18.S096 (Topics in Mathematics with Applications in Finance): `ocw.mit.edu`

Chapter 6

Stochastic Processes in Finance

Learning Objectives

By the end of this chapter, students will be able to:

1. State the Arithmetic Random Walk (ARW), Geometric Random Walk (GRW), and Ornstein-Uhlenbeck (OU) processes in discrete time.
2. Derive the mean and variance of each process at any future date from its one-period recursion.
3. Explain why the GRW requires a convexity adjustment to the drift, and derive that adjustment.
4. Simulate each process in Python and produce confidence bands around forecast paths.
5. Choose the right process for a given financial variable based on whether it can go negative, mean-revert, or experience jumps.
6. Estimate process parameters from historical data using linear regression and method-of-moments.
7. Extend any base process by adding jumps to capture fat-tailed returns.

6.1 Introduction

Chapter 5 taught us how to simulate random variables, single random outcomes at fixed times. But most financial variables evolve over time: stock prices change daily, interest rates drift over years, oil prices move monthly.

A **stochastic process** is a model for how a random variable evolves over time. ("Stochastic" is just another word for "random.") This chapter develops the three core stochastic processes used in modern finance. Each is a discrete-time recursion: given the value today, the process specifies a probability distribution for the value tomorrow. By chaining these one-period steps together, we can generate paths into the future, compute statistics of those paths, and price financial instruments whose cash flows depend on them.

The three processes are:

- **Arithmetic Random Walk (ARW).** The value next period equals the value today plus a constant drift plus a random shock. Linear growth with random deviations. Good for variables that can go negative, cash flows, basis spreads, interest rate differentials.
- **Geometric Random Walk (GRW).** The *logarithm* of the value follows an ARW. Equivalently, returns are normally distributed. The process shows exponential growth, and prices stay positive. The standard model for stocks, indices, and commodity prices.

- **Ornstein-Uhlenbeck (OU).** The mean-reverting process. The value tends to drift back exponentially toward a long-run mean. Similar to the model used for radioactive decay. Good for interest rates, exchange rate differentials, commodity basis spreads, and any variable with a natural equilibrium level.

These three processes are not exhaustive, modern quantitative finance uses dozens of variants, including stochastic volatility models, regime-switching models, and processes driven by Lévy noise. But the three we cover here capture the essential structures, and any more advanced model is best understood as a refinement of one of them.

Connection to Chapter 2. Each stochastic process in this chapter has a *deterministic counterpart* from Chapter 2:

- The **ARW** is the stochastic version of a linearly growing annuity (constant additive growth).
- The **GRW** is the stochastic version of an exponentially growing annuity (constant proportional growth).
- The **OU process** is the stochastic version of a mean-reverting annuity (target-reverting growth).

In each case, the stochastic version adds a normally distributed shock to the deterministic update rule. This deep connection means that everything we learned about valuing the deterministic annuities transfers directly: their *expected* cash flows have the same structure as the deterministic cases.

Connection to Chapter 5. Every simulation in this chapter uses the Python toolkit established in Chapter 5: NumPy for random number generation, vectorized computation, and statistics. The Cholesky decomposition reappears when we simulate paths whose values across time are correlated. No new computational tools are introduced; only new process structures to simulate.

This Chapter Completes Part I

After this chapter, you will have the full simulation toolkit needed for Part II: a way to price any cash flow stream by (1) simulating the underlying financial variables, (2) computing the cash flows along each simulated path, and (3) averaging the discounted cash flows. The remaining chapters of the book apply this toolkit to options, capital budgeting, corporate valuation, and risk management.

6.2 The Arithmetic Random Walk

The Arithmetic Random Walk, abbreviated **ARW**, is the simplest stochastic process in finance. It was used by Louis Bachelier in his 1900 PhD thesis to describe stock prices on the Paris Bourse, the very first mathematical model of stock prices, and arguably the start of mathematical finance as a discipline.

Arithmetic Random Walk (ARW): one-period recursion

$$S_{t+1} = S_t + \mu + \sigma z_{t+1}$$

where μ is the per-period drift, σ is the per-period volatility (standard deviation), and $z_{t+1} \sim N(0,1)$ is a standard normal shock realized at time $t+1$.

Each period's z is independent of all earlier z's. The process is fully specified by three parameters: the starting value S_0, the drift μ, and the volatility σ. The recursion is *Markovian*: tomorrow's value depends only on today's, not on the history.

In words: *the value next period equals the value this period, plus a deterministic drift, plus a random shock.* The drift causes systematic growth (or decay) over time; the shock causes random fluctuation around that trend.

6.2.1 Multi-Period Forecast

Applying the recursion j times forward from time t:

$$S_{t+j} = S_t + j\mu + \sigma(z_{t+1} + z_{t+2} + \cdots + z_{t+j})$$

The sum of j independent standard normals has mean zero and variance j, so it is distributed as $N(0, j)$. Therefore:

Moments of the ARW (conditional on S_t):

$$\mathbb{E}[S_{t+j} \mid S_t] = S_t + j\mu$$

$$\mathrm{Var}(S_{t+j} \mid S_t) = j\sigma^2$$

$$\mathrm{SD}(S_{t+j} \mid S_t) = \sigma\sqrt{j}$$

The mean grows linearly with horizon. The standard deviation grows with the *square root* of the horizon, the $\sqrt{t}$ **rule** famous in finance.

Two consequences worth noting:

- Forecasts become *more uncertain* as the horizon grows. Standard deviation at $t = 20$ is $\sigma\sqrt{20}$, more than $4\times$ the one-period volatility.
- The process can drift arbitrarily far from S_0. There is no bound on how positive or negative it can become.

6.2.2 Covariance Structure

Suppose we want to know the covariance between the process at time $t + j$ and time $t + m$, both conditional on S_t. The two values share their first $\min(j, m)$ shocks, since both contain $z_{t+1}, z_{t+2}, \ldots, z_{t+\min(j,m)}$. Each shared shock contributes σ^2 to the covariance, so:

$$\text{Cov}(S_{t+j},\, S_{t+m} \mid S_t) = \sigma^2 \cdot \min(j, m)$$

This is a foundational result. The covariance grows linearly with the *earlier* of the two times. The Cholesky factorization of the resulting $k \times k$ covariance matrix has a particularly simple form: a lower-triangular matrix of ones, scaled by σ. This means we can equivalently simulate the ARW path by (1) iterating the recursion or (2) generating a single multivariate normal with the appropriate covariance structure. The two methods produce statistically identical paths.

6.2.3 Simulation in Python

Two equivalent ways to simulate an ARW path with the basic recursion. The vectorized cumulative-sum version is faster:

ARW simulation

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters: S0=20, mu=0, sigma=5, 20 periods, 5 paths
S0, mu, sigma = 20.0, 0.0, 5.0
N_periods, N_paths = 20, 5

# Vectorized: each row is a path
np.random.seed(7)
z = np.random.normal(size=(N_paths, N_periods))

# S_t = S_0 + cumulative sum of (mu + sigma*z) over time
increments = mu + sigma * z
S = S0 + np.cumsum(increments, axis=1)
S = np.column_stack([np.full(N_paths, S0), S]) # prepend S_0

# Theoretical confidence band: mean +- 2*sigma*sqrt(t)
times = np.arange(0, N_periods + 1)
upper = S0 + mu*times + 2*sigma*np.sqrt(times)
lower = S0 + mu*times - 2*sigma*np.sqrt(times)
```

Each path is a different realization; with $\mu = 0$, paths drift randomly with no systematic direction. The confidence band $S_0 \pm 2\sigma\sqrt{t}$ expands like a square root, the characteristic widening cone of random walks.

Worked Example: ARW Forecast Moments

Setup. Consider an ARW with $S_0 = 20$, $\mu = 0$, $\sigma = 5$, forecast horizon $j = 20$ periods.

Theoretical moments:

$$\mathbb{E}[S_{20} \mid S_0 = 20] = 20 + 20 \times 0 = 20$$

$$\text{Var}(S_{20} \mid S_0 = 20) = 20 \times 25 = 500$$

$$\text{SD}(S_{20} \mid S_0 = 20) = 5\sqrt{20} \approx 22.36$$

95% range at $t = 20$: approximately $[20 - 44.7,\ 20 + 44.7] = [-24.7,\ 64.7]$.

Empirical verification with 10,000 simulated paths gives mean = 20.33, SD = 21.98, within 2% of theory.

Key observation. Even with zero drift, the variance grows linearly. At $t = 100$, SD would be $5\sqrt{100} = 50$, the process could plausibly wander to ± 100 from $S_0 = 20$. This unboundedness is the ARW's strength (clean linear dynamics) and its weakness (can predict implausibly extreme values).

6.2.4 When the ARW Is Appropriate

Despite Bachelier's original use of the ARW for stock prices, a key flaw makes it unsuitable for stocks: ARW values can become negative. A stock has limited liability, its price cannot go below zero. But over many periods of $\sigma\sqrt{t}$ widening, an ARW path will eventually drift negative. For modeling stocks, prices, or any variable with a hard zero floor, the ARW is the wrong choice.

However, plenty of financial variables can legitimately go negative, and for these the ARW is exactly right:

Variable	Why ARW fits
Corporate cash flows	Can be positive or negative; firms operate at a loss in bad years
WTI-Brent oil basis	Difference between two prices; can flip sign with regional supply shocks
Bond yield spreads	Difference between corporate and Treasury yields; can compress to near zero
Trading P&L (mark-to-market)	Gains and losses; no zero bound
Interest rate basis (LIBOR-OIS)	Differential between two rates; can be negative

Corporate Application: ARW for Corporate Cash Flow Forecasting

A corporate planner forecasting next-quarter EBITDA might use an ARW with the firm's historical mean and standard deviation. Suppose the firm has trailing-twelve-month EBITDA of \$100M, with quarterly drift $\mu = \$5M$ (growing) and volatility $\sigma = \$20M$.

Four quarters forward, expected EBITDA is $100 + 4 \times 5 = \$120M$, with standard deviation $20\sqrt{4} = \$40M$. The 95% confidence interval is roughly $[40, 200]$, a wide band reflecting genuine uncertainty in corporate earnings.

For valuation, we don't care only about the expected value: we care about the entire distribution. If EBITDA falls below a covenant threshold (say, \$40M), the firm faces lender consequences. Simulating the ARW path and counting the fraction of paths that breach the covenant gives the probability of distress, a quantitative input to risk-managed capital structure decisions.

6.3 The Geometric Random Walk

Stock prices have two features that make the ARW inappropriate. First, they cannot go negative, limited liability puts a hard floor at zero. Second, they grow *proportionally*, not additively: a \$10 stock and a \$1000 stock have roughly comparable percentage returns, not comparable dollar changes.

The Geometric Random Walk addresses both problems by modeling the *logarithm* of the price as an ARW. Since the log of any positive number can take any value, the price itself, the exponential of the log, is always positive. And additive movements in log-prices correspond to proportional movements in prices.

Geometric Random Walk (GRW): one-period recursion

$$S_{t+1} = S_t \cdot \exp\left[(\alpha - \tfrac{1}{2}\sigma^2) + \sigma z_{t+1}\right]$$

or equivalently, in log form:

$$\ln(S_{t+1}) = \ln(S_t) + (\alpha - \tfrac{1}{2}\sigma^2) + \sigma z_{t+1}$$

where α is the per-period expected return, σ is the per-period log-return volatility, and $z_{t+1} \sim N(0, 1)$.

The strange-looking $-\frac{1}{2}\sigma^2$ term in the exponent is essential. It is called the **convexity correction**, and without it the process would deliver the wrong expected return. We will derive it carefully, it is one of the few places in finance where the algebra has both elegance and substance.

6.3.1 Why the Convexity Correction?

The source of the bias is **Jensen's inequality**: for a convex function f and a random variable X, $f(E[X]) \leq E[f(X)]$. Since $\exp(\cdot)$ is convex everywhere, applying it to mean-zero noise and then averaging gives strictly more than exponentiating the mean directly. Figure 6.1 shows this geometrically with the simplest possible example, $X = \pm 1$ equally likely: $f(E[X]) = \exp(0) = 1$ sits on the curve itself, while $E[f(X)]$, the average of the two outcomes, sits on the chord connecting them, strictly above the curve because the curve bows below any chord joining two of its points. The lemma below quantifies that gap exactly for our actual noise term, σz.

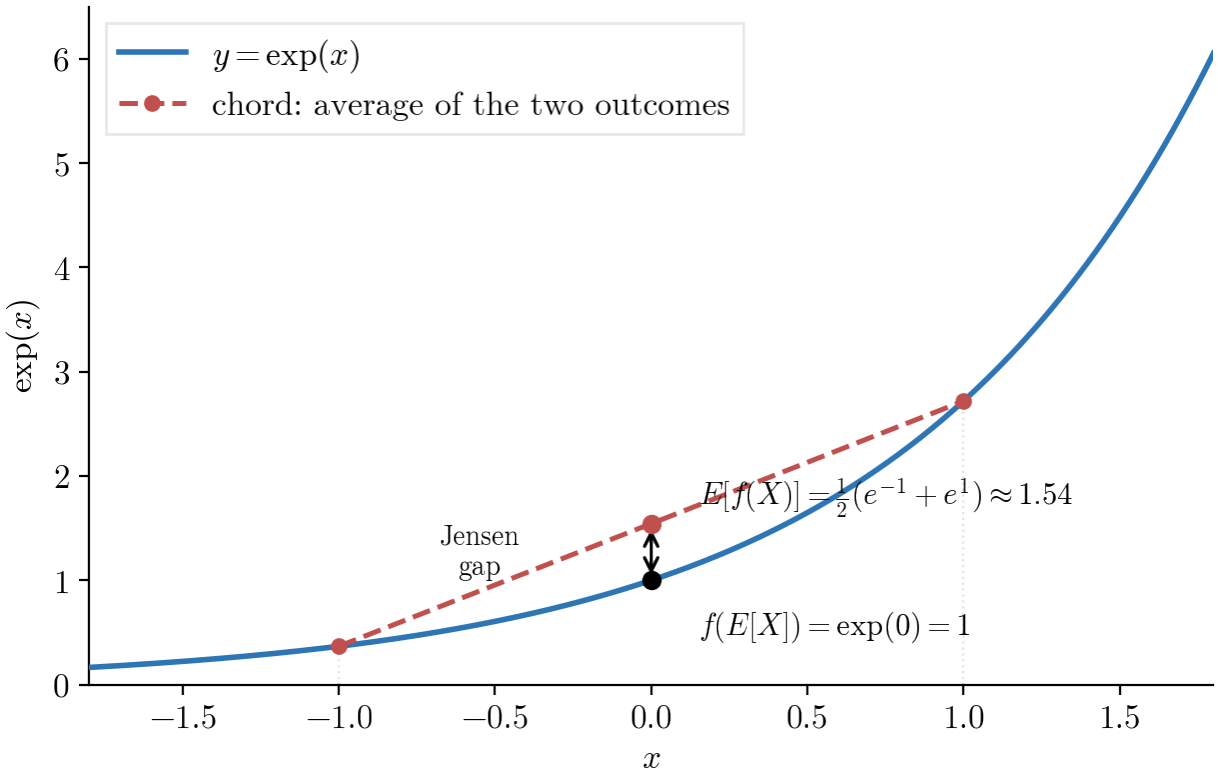


Figure 6.1: *Jensen's inequality applied to* $\exp(\cdot)$*, for the minimal example* $X = \pm 1$ *with equal probability.* $f(E[X]) = \exp(0) = 1$ *lies on the curve;* $E[f(X)] = \frac{1}{2}(e^{-1} + e^{1}) \approx 1.54$ *lies on the chord connecting the two sampled outcomes. The vertical gap between them is the convexity bias that the drift correction below exists to cancel.*

Math Review: Deriving the $(\alpha - \frac{1}{2}\sigma^2)$ Adjustment

We want our process to satisfy $\mathbb{E}[S_{t+1} \mid S_t] = S_t \cdot e^{\alpha}$, i.e., the expected return is exactly α (in continuous compounding). Without the correction, we might naively write:

$$S_{t+1} = S_t \cdot \exp(\alpha + \sigma z)$$

To compute the expectation, we use a key result about the lognormal distribution.

Lemma. If $Z \sim N(0,1)$, then for any constant a:

$$\mathbb{E}[\exp(aZ)] = \exp(\tfrac{1}{2}a^2)$$

Proof. Compute the integral directly:

$$\mathbb{E}[\exp(aZ)] = \int \exp(az) \cdot \frac{1}{\sqrt{2\pi}} \exp(-z^2/2)\,\mathrm{d}z$$

Complete the square in the exponent: $az - z^2/2 = -\frac{1}{2}(z-a)^2 + \frac{1}{2}a^2$. Then:

$$\mathbb{E}[\exp(aZ)] = \exp(\tfrac{1}{2}a^2) \cdot \int \frac{1}{\sqrt{2\pi}} \exp(-\tfrac{1}{2}(z-a)^2)\,\mathrm{d}z = \exp(\tfrac{1}{2}a^2)$$

The remaining integral is the density of $N(a,1)$ integrated over its entire support, which equals 1. ■

Apply to the GRW. With $a = \sigma$, the lemma gives:

$$\mathbb{E}[\exp(\sigma z)] = \exp(\tfrac{1}{2}\sigma^2)$$

So if we had used the naive form without the correction:

$$\mathbb{E}[S_{t+1}] = S_t \cdot e^{\alpha} \cdot \mathbb{E}[\exp(\sigma z)] = S_t \cdot e^{\alpha} \cdot e^{(\sigma^2/2)} = S_t \cdot e^{(\alpha + \sigma^2/2)}$$

The expected return would be $\alpha + \frac{1}{2}\sigma^2$, *not* α. The convexity of the exponential function inflates the mean by $\frac{1}{2}\sigma^2$.

The fix. Replace the naive drift α with $\alpha - \frac{1}{2}\sigma^2$. Then:

$$\mathbb{E}[S_{t+1}] = S_t \cdot \exp(\alpha - \tfrac{1}{2}\sigma^2 + \tfrac{1}{2}\sigma^2) = S_t \cdot e^{\alpha} \;\checkmark$$

The correction exactly cancels the convexity inflation, restoring the desired expected return.

Numerical illustration. Suppose $S_0 = \$100$, $\alpha = 10\%$, $\sigma = 30\%$, $T = 1$ year. The expected price one year out should be $100 \times e^{0.10} = \$110.52$.

Simulating with the correction ($\alpha - \frac{1}{2}\sigma^2$): empirical $\mathbb{E}[S_T] = \$110.54$. ✓

Simulating without correction (just α): empirical $\mathbb{E}[S_T] = \$115.63$. Bias = \$5.09. The process delivers an expected return of 14.6%, not the intended 10%.

This 5% discrepancy compounds over time. Over 10 years, the uncorrected process would predict $\mathbb{E}[S_T] \approx 159$ instead of the correct $100 \times e^{1.0} = 272$, a 41% understatement. The convexity correction is essential for any quantitative work with the GRW.

6.3.2 Multi-Period Moments

To project j periods forward, iterate the GRW recursion. Since log-returns are independent and normal:

$$\ln(S_{t+j}) = \ln(S_t) + j(\alpha - \tfrac{1}{2}\sigma^2) + \sigma(z_{t+1} + \cdots + z_{t+j})$$

The sum of j independent standard normals is $N(0, j)$:

Moments of $\log(S)$ under GRW:

$$\mathbb{E}[\ln(S_{t+j}) \mid S_t] = \ln(S_t) + j(\alpha - \tfrac{1}{2}\sigma^2)$$

$$\mathrm{Var}(\ln(S_{t+j}) \mid S_t) = j\sigma^2$$

And by the lognormal moment formula:

$$\mathbb{E}[S_{t+j} \mid S_t] = S_t \cdot \exp(j\alpha)$$

Notice that $\ln(S)$ follows an ARW with drift $(\alpha - \frac{1}{2}\sigma^2)$ and volatility σ. The GRW is just an ARW on the log scale.

6.3.3 Simulation in Python

GRW simulation

```
import numpy as np

S0, alpha, sigma = 1300.0, 0.10, 0.18 # S&P-like parameters
N_years, periods_per_year = 2, 12
N = N_years * periods_per_year
dt = 1.0 / periods_per_year

# Per-period parameters (annual rates -> monthly)
log_drift = (alpha - 0.5 * sigma**2) * dt
log_vol = sigma * np.sqrt(dt)

np.random.seed(42)
N_sim = 50_000
z = np.random.normal(size=(N_sim, N))
log_S = np.log(S0) + np.cumsum(log_drift + log_vol * z, axis=1)
S_paths = np.exp(log_S)

# Empirical vs. theoretical E[S_T]
print(f"Theoretical E[S_T] = S0 * exp(alpha*T) = "
      f"{S0 * np.exp(alpha * N_years):.2f}")
print(f"Empirical E[S_T] = {S_paths[:, -1].mean():.2f}")

# Probability that the index exceeds 2000 anytime in 24 months
max_per_path = S_paths.max(axis=1)
```

```
prob = (max_per_path > 2000).mean()
print(f"P(max > 2000 within 24 months) = {prob:.4f}")
```

Output

```
Theoretical E[S_T] = S0 * exp(alpha*T) = 1587.82
Empirical E[S_T] = 1585.55
P(max > 2000 within 24 months) = 0.1990
```

Time conversion. Annual parameters convert to per-period parameters as follows. If α is the annual continuously-compounded expected return and σ is annual volatility, then per period of length Δt:

$$\alpha_{period} = \alpha \cdot \Delta t$$

$$\sigma_{period} = \sigma \cdot \sqrt{\Delta t}$$

This is the famous $\sqrt{t}$ **rule of volatility scaling.** Volatility scales with $\sqrt{\Delta t}$ while drift scales with Δt. Over short horizons, the random component dominates; over long horizons, the drift dominates.

Limitations of the GRW for Real Stock Returns

The GRW is the workhorse model for stock prices and is built into nearly every option pricing model. However, real stock returns deviate from the GRW assumptions in two important ways:

- Real returns are **negatively skewed**: sudden price drops are more common than the symmetric normal distribution predicts.
- Real returns are **leptokurtic** (have fat tails): extreme moves, both up and down, happen more often than the normal distribution allows.

During the 1987 market crash, the S&P 500 fell about 23 standard deviations in a single day. Under a normal distribution that event has probability $\approx 10^{-115}$, it should never happen in the history of the universe. But it did. Real returns have much fatter tails than the GRW admits.

More sophisticated models extend the GRW to capture these effects, stochastic volatility, regime switching, and the jumps we cover in Section 6.6. Even with these extensions, real return distributions remain harder to model than any clean theory admits.

6.4 Mean Reversion: The Ornstein-Uhlenbeck Process

Some financial variables don't drift indefinitely. Interest rates fluctuate around an equilibrium set by central bank policy and long-run macroeconomic conditions. The WTI-Brent oil basis varies

but rarely strays far from zero in the long run (arbitrageurs move oil between regions when the gap grows wide). Exchange rates between developed countries are mean-reverting in the very long run, even if not in the short run.

For such variables, the ARW and GRW both miss an essential feature: their variance grows unboundedly with horizon, when in reality these variables are bounded by economic forces that pull them back toward equilibrium. The **Ornstein-Uhlenbeck (OU) process**, named for the physicists Leonard Ornstein and George Uhlenbeck, who introduced its mean-reverting structure in 1930, captures this behavior.

Ornstein-Uhlenbeck (OU) process: one-period recursion

$$S_{t+1} = S_t + \kappa(\mu - S_t) + \sigma z_{t+1}$$

where μ is the **long-run mean**, $\kappa \in (0, 1)$ is the **speed of mean reversion**, σ is the per-period volatility, and $z_{t+1} \sim N(0, 1)$.

The intuition is direct. The term $\kappa(\mu - S_t)$ is a deterministic pull toward μ:

- If $S_t > \mu$ (above the long-run mean), the drift $\kappa(\mu - S_t)$ is negative, the process is pulled down.
- If $S_t < \mu$, the drift is positive, the process is pulled up.
- If $S_t = \mu$, the drift is zero; only the random shock moves the process.

The speed κ controls how quickly the pull operates. With $\kappa = 0$, there is no mean reversion, the OU collapses to an ARW. With $\kappa = 1$, the process snaps to μ in one period plus noise. Typical values are 0.05 to 0.5 per period for slow to moderate mean reversion.

6.4.1 Multi-Period Forecast

To project the OU process forward, iterate the recursion. Let $D_t = S_t - \mu$ be the deviation from the long-run mean. Then $D_{t+1} = (1 - \kappa)D_t + \sigma z_{t+1}$, a first-order autoregression in D.

An autoregression predicts a variable from its own past rather than another variable; *first-order* means only the most recent lag matters, and $(1 - \kappa)$ below 1 in magnitude is what keeps D mean-reverting instead of drifting like a random walk.

Iterating:

$$D_{t+j} = (1 - \kappa)^j D_t + \sigma \sum_{i=1}^{j} (1 - \kappa)^{j-i} z_{t+i}$$

The gap $j - i$ sets the impact: a shock right at the forecast date ($j - i$ near 0) keeps close to full weight, while one far in the past ($j - i$ large) has been mean-reverted away almost entirely, since $(1 - \kappa)^{j-i}$ shrinks as that gap grows.

Since $S_{t+j} = \mu + D_{t+j}$, adding μ back to the deviation recovers the moments of S_{t+j} shown below:

Moments of the OU process (conditional on S_t):

$$\mathbb{E}[S_{t+j} \mid S_t] = \mu + (S_t - \mu)(1-\kappa)^j$$

$$\text{Var}(S_{t+j} \mid S_t) = \sigma^2 \cdot \frac{1-(1-\kappa)^{2j}}{\kappa(2-\kappa)}$$

As $j \to \infty$:

$$\mathbb{E}[S_\infty] = \mu, \qquad \text{Var}(S_\infty) = \frac{\sigma^2}{\kappa(2-\kappa)}$$

The **long-run variance** is finite and bounded, the defining feature of mean reversion.

Compare this to the ARW, whose variance grows as $j\sigma^2$ without bound. The OU process has a stable long-run distribution; the ARW does not.

Worked Example: OU Long-Run Distribution

Setup. Oil price modeled as OU with $\mu = \$80$/bbl, $\kappa = 0.20$ per month, $\sigma = \$5$/bbl per month. Currently $S_0 = \$100$ (above the long-run mean).

One-month forecast.

$$\mathbb{E}[S_1 \mid S_0] = 80 + (100-80)(1-0.20) = 80 + 16 = \$96$$

$$\text{SD}(S_1 \mid S_0) = 5$$

Each month, the expected price falls by 20% of the gap to \$80.

One-year forecast.

$$\mathbb{E}[S_{12} \mid S_0] = 80 + 20 \cdot (0.80)^{12} = 80 + 20 \cdot 0.0687 = \$81.37$$

After 12 months, the process has decayed 93% of the way to the long-run mean.

Long-run variance.

$$\text{Var}(S_\infty) = 25/(0.20 \cdot 1.80) = 25/0.36 = 69.44$$

$$\text{SD}(S_\infty) = \sqrt{69.44} = \$8.33$$

The long-run distribution is approximately normal with mean \$80 and standard deviation \$8.33. The 95% range is roughly \$63.34 to \$96.66.

Key insight. Even though the one-period volatility is \$5, the long-run dispersion is only \$8.33, not $\$5 \times \sqrt{12} = \$17.32$ as the ARW would predict, and not unbounded as time grows. The mean reversion keeps the variance contained.

6.4.2 Covariance Structure

For two future times $t + j$ and $t + m$ with $m > j$:

$$\mathrm{Cov}(S_{t+j},\, S_{t+m} \mid S_t) = \mathrm{Var}(S_{t+j} \mid S_t) \cdot (1 - \kappa)^{m-j}$$

The covariance decays exponentially as the time gap grows. The autocorrelation function of an OU process is $\rho(h) = (1 - \kappa)^h$, so distant observations become uncorrelated. This is in sharp contrast to the ARW, where $\mathrm{Cov}(S_{t+j},\, S_{t+m}) = \sigma^2 \cdot \min(j, m)$ grows without bound.

6.4.3 Simulation in Python

OU simulation

```python
import numpy as np

# Oil price example: mu_lr=80, kappa=0.20, sigma=5, S0=100
S0, mu_lr, kappa, sigma = 100.0, 80.0, 0.20, 5.0
N_periods = 24 # 2 years monthly
N_paths = 1000

np.random.seed(0)
z = np.random.normal(size=(N_paths, N_periods))
S = np.zeros((N_paths, N_periods + 1))
S[:, 0] = S0
for t in range(N_periods):
    S[:, t+1] = S[:, t] + kappa * (mu_lr - S[:, t]) + sigma * z[:, t]

# Verify at t=12 against the analytic formula
t_check = 12
emp_mean = S[:, t_check].mean()
emp_var = S[:, t_check].var()
theo_mean = mu_lr + (S0 - mu_lr) * (1 - kappa)**t_check
theo_var = sigma**2 * (1 - (1 - kappa)**(2*t_check)) / (kappa * (2 - kappa))

print(f"At t={t_check}:")
print(f" Empirical: mean={emp_mean:.2f}, var={emp_var:.2f}")
print(f" Theory: mean={theo_mean:.2f}, var={theo_var:.2f}")
```

Output

```
At t=12:
  Empirical: mean=81.42, var=69.25
  Theory: mean=81.37, var=69.12
```

6.4.4 When the OU Process Is Appropriate

The OU process is the right choice when the variable has a *natural equilibrium level* pulled toward by economic forces. Examples:

Variable	Why OU fits	Typical κ (monthly)
WTI-Brent oil basis	Arbitrage between regions pulls prices together	0.1-0.3
Short-term interest rates	Central bank policy anchors around a target	0.02-0.08
Real interest rates	Real returns to capital are stable in long run	0.01-0.04
Commodity prices	Supply-demand equilibrium with shocks	0.02-0.10
Volatility (VIX)	Reverts to long-run average $\approx 18\%$	0.15-0.40
Currency basis swaps	Funding costs equilibrate	0.05-0.20

Mean Reversion Implies Predictability

Because the OU drift is $\kappa(\mu - S_t)$, we can predict future values better than a coin flip would: when $S_t > \mu$ we predict decline, when $S_t < \mu$ we predict increase. This is fine for variables like oil basis or interest rates where there's no immediate trading mechanism to enforce equality. But for **freely traded asset prices**, stocks, indices, liquid futures, mean reversion would imply free profits. If everyone believed prices would fall, they would sell now, and prices would fall now. Markets enforce the Efficient Market Hypothesis: freely traded prices cannot have predictable mean-reverting drift. For those, use the GRW.

6.5 Parameterization and Estimation

Each stochastic process has parameters: ARW has drift and volatility, GRW has expected return and volatility, OU has long-run mean, mean-reversion speed, and volatility. To use these models in practice, we must estimate the parameters from historical data.

This section shows how to do so. The methods are nothing more than ordinary linear regression and method of moments, ideas familiar from any statistics course, applied to the specific structure of each process.

6.5.1 ARW Parameters: μ and σ

Given a historical series $S_0, S_1, \ldots, S_T$, compute the first differences $\Delta S_t = S_t - S_{t-1}$. Under the ARW, these differences are iid normal with mean μ and standard deviation σ. So:

$$\hat{\mu} = \textbf{\textit{sample mean of }} \Delta S_t$$
$$\hat{\sigma} = \textbf{\textit{sample standard deviation of }} \Delta S_t$$

ARW parameter estimation

```
import numpy as np

# Suppose S is a NumPy array of historical observations
delta_S = np.diff(S)
mu_hat = delta_S.mean()
sigma_hat = delta_S.std(ddof=1)
print(f"Drift: {mu_hat:.4f}, Volatility: {sigma_hat:.4f}")
```

6.5.2 GRW Parameters: α and σ

Take logs first. Compute log-returns $r_t = \ln(S_t/S_{t-1}) = \ln(S_t) - \ln(S_{t-1})$. Under the GRW, these are iid normal with mean $(\alpha - \frac{1}{2}\sigma^2)$ and standard deviation σ. So:

$\hat{\sigma} = \textbf{\textit{sample standard deviation of log-returns}}$
$\hat{\alpha} = \textbf{\textit{sample mean of log-returns}} + \frac{1}{2}\hat{\sigma}^2$
The convexity correction reappears: to recover α we must add back $\frac{1}{2}\sigma^2$.

GRW parameter estimation

```
log_returns = np.diff(np.log(S))
sigma_hat = log_returns.std(ddof=1)
mean_log = log_returns.mean()
alpha_hat = mean_log + 0.5 * sigma_hat**2
print(f"alpha: {alpha_hat:.4f}, sigma: {sigma_hat:.4f}")
```

Time scaling. If you estimate from *monthly* data, your parameters are monthly. To annualize:

$$\alpha_{annual} = 12 \cdot \alpha_{monthly}$$

$$\sigma_{annual} = \sqrt{12} \cdot \sigma_{monthly}$$

Drift scales linearly with time; volatility scales with the square root. This is the same $\sqrt{t}$ rule from Section 6.2.

6.5.3 OU Parameters: μ, κ, and σ

The OU process has more structure, so we estimate via regression. Rearrange the recursion:

$$\Delta S_t = S_t - S_{t-1} = \kappa(\mu - S_{t-1}) + \sigma z_t = \kappa\mu - \kappa S_{t-1} + \sigma z_t$$

This is a linear regression of ΔS_t on S_{t-1}:

$$\Delta S_t = a + b \cdot S_{t-1} + \varepsilon_t$$

with intercept $a = \kappa\mu$, slope $b = -\kappa$, and residual standard deviation σ. Run the regression and back out the OU parameters:

$$\hat{\kappa} = -\hat{b}$$
$$\hat{\mu} = \hat{a}/\hat{\kappa} = \hat{a}/(-\hat{b})$$
$$\hat{\sigma} = \textbf{\textit{residual standard deviation}}$$

OU parameter estimation

```
import numpy as np

# S is the historical series
dS = np.diff(S)
S_lag = S[:-1]

# Linear regression: dS = a + b * S_lag + epsilon
A = np.column_stack([np.ones(len(S_lag)), S_lag])
coef, _, _, _ = np.linalg.lstsq(A, dS, rcond=None)
intercept, slope = coef

# Back out OU parameters
kappa_hat = -slope
mu_hat = intercept / kappa_hat
residuals = dS - A @ coef
sigma_hat = residuals.std(ddof=2) # 2 parameters fitted

# Test statistical significance of mean reversion: H0 = "no mean reversion"
# (slope = 0). If t-stat on slope is significant and negative, we have evidence
# of mean reversion.
n = len(S_lag)
se_slope = sigma_hat / np.sqrt(((S_lag - S_lag.mean())**2).sum())
t_stat = slope / se_slope
```

```
print(f"Estimated kappa: {kappa_hat:.4f}")
print(f"Estimated mu: {mu_hat:.4f}")
print(f"Estimated sigma: {sigma_hat:.4f}")
print(f"t-stat on slope: {t_stat:.2f} "
      f"(significant if |t| > 2)")
```

Testing for mean reversion. The regression also provides a test of whether mean reversion is present at all. If the t-statistic on the slope is *statistically insignificant* ($|t| < 2$ roughly), we cannot reject $\kappa = 0$, the variable may be a random walk rather than mean-reverting. This is the classic **Dickey-Fuller test** for stationarity, presented here in its simplest form.

Worked Example: Estimating OU Parameters from Synthetic Data

Generate 240 monthly observations of an OU process with known parameters, then estimate them. This is a sanity check: the regression should recover the true parameters within sampling noise.

OU estimation example

```
# Generate synthetic OU data
np.random.seed(123)
true_kappa, true_mu, true_sigma = 0.15, 50.0, 5.0
N = 240 # 20 years monthly
S = np.zeros(N)
S[0] = 60.0
for t in range(1, N):
    S[t] = S[t-1] + true_kappa*(true_mu - S[t-1]) + true_sigma*np.random.normal()

# Run regression to estimate parameters
dS = np.diff(S)
S_lag = S[:-1]
A = np.column_stack([np.ones(len(S_lag)), S_lag])
coef, _, _, _ = np.linalg.lstsq(A, dS, rcond=None)
intercept, slope = coef
kappa_hat = -slope
mu_hat = intercept / kappa_hat
sigma_hat = (dS - A @ coef).std(ddof=2)

print(f"True: kappa=0.150, mu=50.00, sigma=5.00")
print(f"Estimated: kappa={kappa_hat:.3f}, "
      f"mu={mu_hat:.2f}, sigma={sigma_hat:.2f}")
```

Output

```
True: kappa=0.150, mu=50.00, sigma=5.00
Estimated: kappa=0.159, mu=49.70, sigma=5.14
```

With 20 years of monthly data, we recover κ to within 6%, μ to within 1%, and σ to within 3%. Shorter samples or higher noise levels would give noisier estimates; longer samples would tighten the fit further. Real-world data is more challenging because the true parameters may drift over time, a topic for time-varying parameter models.

6.6 Jump Processes

Real financial variables sometimes move suddenly and far. A merger announcement doubles a stock overnight. A pandemic crashes oil prices 30% in days. An interest rate decision moves the entire yield curve in seconds. Such events are not well captured by the smooth normal increments of an ARW, GRW, or OU process. Sample sizes of even decades may not contain enough "normal" returns to estimate true volatility correctly, because the tail observations are dominated by jumps.

The fix is to overlay **jumps** on top of any base process. We assume that in any single period, there is a small probability of a discrete jump in addition to the usual continuous evolution.

6.6.1 Compound Poisson Jumps

The standard model is the **compound Poisson process**. In any one period:

- With probability $1-p$, no jump occurs. The base process (ARW, GRW, OU) evolves normally.
- With probability p, a jump occurs. The size of the jump is itself a random variable, typically drawn from a normal distribution $N(\mu_J, \sigma_J^2)$.

Compound Poisson jump in a GRW (multiplicative):

$$S_{t+1} = S_t \cdot \exp\left[(\alpha - \tfrac{1}{2}\sigma^2) + \sigma z_{t+1}\right] \cdot J_{t+1}$$

where the jump multiplier J_{t+1} equals 1 if no jump occurs and $\exp(\mu_J + \sigma_J Y)$ with $Y \sim N(0,1)$ if a jump occurs.

Compound Poisson jump in an ARW or OU (additive):

$$S_{t+1} = (\textbf{\textit{base recursion}}) + J_{t+1}$$

where J_{t+1} equals 0 with probability $1-p$ and a normal random variable with probability p.

6.6.2 Simulation in Python

Compound Poisson jumps on OU

```
# Oil shock example: 1% chance per month of a 100% price spike
# Base: OU with mu_lr=80, kappa=0.20, sigma=5
# Jump: with prob p=0.01 per month, multiply by exp(N(0.69, 0.1))
# (i.e., approximately doubling with some noise)
import numpy as np

S0, mu_lr, kappa, sigma = 80.0, 80.0, 0.20, 5.0
p_jump = 0.01
mu_J, sigma_J = np.log(2), 0.10 # log-jump centered at ln(2) (doubling)
N = 60 # 5 years monthly
N_paths = 5

np.random.seed(11)
S = np.zeros((N_paths, N + 1))
S[:, 0] = S0
for t in range(N):
    z = np.random.normal(size=N_paths)
    # Base OU step
    S[:, t+1] = S[:, t] + kappa * (mu_lr - S[:, t]) + sigma * z
    # Jump overlay (additive in OU, but applied as a multiplicative shock)
    jumps_occur = np.random.uniform(size=N_paths) < p_jump
    jump_sizes = np.exp(mu_J + sigma_J * np.random.normal(size=N_paths))
    S[jumps_occur, t+1] *= jump_sizes[jumps_occur]

print("Five sample paths, final price:")
for i, val in enumerate(S[:, -1]):
    print(f" Path {i+1}: ${val:.2f}")
```

Output

```
Five sample paths, final price:
  Path 1: $76.70
  Path 2: $77.64
  Path 3: $84.52
  Path 4: $68.38
  Path 5: $70.01
```

In any single 5-year window, most of 5 simulated paths will look like an ordinary OU process. A few will contain one or two jumps, producing visible vertical leaps. After a jump, the OU process pulls back toward the long-run mean over the following months, jumps are temporary in mean-reverting processes.

In an ARW or GRW, by contrast, **jumps are permanent**: with no mean reversion, there is no

force to pull the process back. A stock that gaps up 20% on a merger announcement stays roughly at the new level.

Calibrating jumps in practice. Estimating jump parameters from data is delicate. Distinguishing a "jump" from an unusually large normal return requires statistical tests beyond this chapter, typically nonparametric tests using high-frequency intraday data. For applied work, jump parameters are often calibrated from option market prices (using implied volatility skew) or set judgmentally based on historical extreme events. The point of including jumps is not to fit history precisely, but to *acknowledge that they can happen* and incorporate the resulting fat tails into risk and valuation analyses.

6.7 Choosing a Process

With three core processes and the option to add jumps, how do we choose? The decision turns on three diagnostic questions about the financial variable:

Decision tree for stochastic process selection

Q1. Can the variable take negative values?

- **Yes** (cash flows, basis spreads, P&L, rate differentials) → go to Q2.
- **No** (stock prices, indices, commodity prices) → use the **GRW**. Skip to Q3 for jumps.

Q2. Does the variable have a long-run equilibrium level?

- **Yes** (interest rates, oil basis, volatility around its mean) → use the **OU process**.
- **No** (trading P&L, EBITDA over short horizons) → use the **ARW**.

Q3. Do discrete shocks happen?

- **Yes** (announcements, regime changes, supply disruptions, geopolitical events) → add compound Poisson jumps.
- **No** (smooth, well-behaved variable) → base process alone is sufficient.

Several variables and their typical stochastic process specification, as a reference:

Variable	Process	Notes
Stock price (single name)	GRW + jumps	Mergers, earnings surprises
Equity index (S&P 500)	GRW + jumps	Crashes are jumps; trend is GRW
Crude oil price	GRW	Mostly multiplicative growth

Variable	Process	Notes
WTI-Brent basis	OU	Mean-reverts toward zero
10-year Treasury yield	OU	Anchored by Fed policy
LIBOR-OIS spread	OU + jumps	Spikes during stress events
Corporate EBITDA	ARW	Can go negative, no equilibrium
Corporate revenue (growing firm)	GRW	Multiplicative growth, positive
FX rate (USD/EUR)	GRW	Approximately random walk
Exchange rate (emerging mkt)	GRW + jumps	Devaluations are jumps
VIX (equity volatility)	OU + jumps	Mean-reverts to ≈ 18, spikes in crises
GDP growth rate	OU	Mean-reverts toward long-run growth

Personal Finance: Choosing a Process for Retirement Modeling

In retirement planning, you simulate future market returns to estimate the probability your savings will last. Choose processes for each input:

- Stock returns: **GRW + jumps**. Historical S&P 500 returns are roughly $\alpha = 8\%$, $\sigma = 16\%$ annual, with rare jump events (1929, 1987, 2008, 2020).
- Bond returns: **GRW** with lower drift and volatility ($\alpha = 4\%$, $\sigma = 5\%$).
- Inflation: **OU** toward 2.5% with $\kappa \approx 0.10$/yr and $\sigma \approx 1.5\%$.
- Wage growth: **OU** toward 3.5% with $\kappa \approx 0.15$/yr.
- Healthcare costs: **GRW** with drift 5-7% (faster than general inflation).

With these specifications and the Cholesky-based multivariate simulation from Chapter 5, you can produce thousands of plausible retirement trajectories and compute the probability of running out of money at various withdrawal rates. This is exactly the methodology used by financial planners and academic studies of safe withdrawal rates.

6.8 Connection to Cash Flow Valuation

Throughout Part I, we have valued cash flows whose patterns were known: level annuities, growing annuities, mean-reverting annuities, fixed coupon bonds, defaultable bonds with constant hazard rates. The GVE, take expected cash flows, discount each by the appropriate factor, sum them, produced exact values.

The simulation toolkit of Chapter 5 combined with the stochastic processes of this chapter extend our reach to *any cash flow stream whose path matters.* The general procedure:

General simulation-based valuation procedure:

1. Identify the underlying random factors (stock price, interest rate, oil price, etc.).
2. Choose a stochastic process for each factor using the Section 6.7 decision tree.
3. Estimate parameters from historical data (Section 6.5) or market prices.
4. Simulate N paths of the factor(s) using the methods of Chapter 5.
5. For each path, compute the cash flows specified by the contract or asset.
6. Discount each path's cash flows at the appropriate rate.
7. Average across paths to get the valuation. Quantify error via standard error.

This procedure handles essentially any cash flow valuation problem that the GVE alone cannot. Examples:

- **Path-dependent cash flows.** A contract that pays based on the maximum stock price over a year requires simulating the entire path, not just the terminal value.
- **Correlated cash flows.** A natural-gas producer's revenue depends on both production volume and price, which may be correlated with weather and macroeconomic factors. Simulate the joint distribution of all relevant variables.
- **Real options.** A mining project the owner can choose to begin or abandon depending on commodity prices. The optimal decision rule, "begin operations if and only if price exceeds threshold p^*", must be evaluated path by path.
- **Insurance and pension liabilities.** Long-horizon obligations that depend on inflation, interest rates, and longevity. Simulate the joint evolution of all three.

The catch: discount rate. Throughout this chapter we have been silent on what discount rate to use when valuing simulated cash flows. For risk-free cash flows (Treasuries, government-guaranteed), the answer is the risk-free yield curve. For risky cash flows (corporate, equity, real assets), the right rate depends on the risk of the cash flow, a topic we will address head-on in Chapter 7, which begins Part II.

Until then, simulation gives us *expected* cash flows; pricing those expected cash flows requires a theory of how investors price risk. The next chapter delivers that theory, building on the Capital Asset Pricing Model (CAPM) and its extensions to derive the cost of capital that closes the GVE.

6.9 Chapter Summary

Three stochastic processes capture the essential dynamics of nearly every financial variable:

Process	Best for	Key formula
ARW	Cash flows, basis spreads	$S_{t+1} = S_t + \mu + \sigma z$
GRW	Stocks, indices, commodity prices	$S_{t+1} = S_t \cdot \exp[(\alpha - \frac{1}{2}\sigma^2) + \sigma z]$
OU	Interest rates, basis, volatility	$S_{t+1} = S_t + \kappa(\mu - S_t) + \sigma z$

Three takeaways:

- **Variance scales with horizon differently for different processes.** ARW: linear. GRW: linear in log-variance. OU: bounded.
- **The GRW requires a convexity adjustment** $(\alpha - \frac{1}{2}\sigma^2)$ to deliver the intended expected return. Skip this adjustment and your simulations will be biased upward.
- **Mean reversion (OU) implies predictability.** This is fine for variables anchored by economic forces (interest rates, basis), but freely-traded asset prices *must not* exhibit mean reversion or arbitrage profits would emerge.

Part I complete. With Chapters 1-6, we have the full machinery to value known cash flows under any pattern, derive market parameters from observed prices, simulate uncertain future scenarios, and characterize the dynamics of financial variables. Part II addresses the central remaining question: how do investors price risk, and what discount rate should we use for risky cash flows?

6.10 Key Terms

Term	Definition
Arithmetic Random Walk (ARW)	Stochastic process $S_{t+1} = S_t + \mu + \sigma z$; linear drift with additive normal shocks; can go negative.
Compound Poisson process	A process with discrete random jumps occurring at random times; used to model sudden price movements.
Convexity correction	The $(\alpha - \frac{1}{2}\sigma^2)$ adjustment to the GRW drift that ensures $\mathbb{E}[S_{t+1}] = S_t \cdot e^{\alpha}$.
Geometric Random Walk (GRW)	Stochastic process where $\ln(S)$ follows an ARW; multiplicative growth; prices stay positive.
Jump process	A stochastic process with discrete random shocks in addition to the usual normal increments of the base process.
Long-run mean (μ)	Equilibrium level toward which an OU process drifts.

Term	Definition
Markov process	A process whose next value depends only on the current value, not the full history.
Mean reversion speed (κ)	Fraction of the gap ($\mu - S_t$) eliminated per period in an OU process.
Ornstein-Uhlenbeck (OU)	Mean-reverting stochastic process; variance grows toward a bounded long-run level.
Random walk	Discrete-time stochastic process with iid increments.
Stochastic process	Mathematical model for the evolution of a random variable over time.
Square-root scaling	Volatility scales with $\sqrt{\Delta t}$ while drift scales with Δt.

6.11 Interview Questions

Answer each in 2-3 sentences. Show familiarity with terminology, key formulas, and the conceptual reasoning behind them.

1. Explain the Arithmetic Random Walk in your own words. Why is its variance proportional to horizon?
2. Why is the ARW unsuitable for modeling stock prices? Give a specific example of a variable where it works well.
3. Derive the convexity correction ($\alpha - \frac{1}{2}\sigma^2$) in the Geometric Random Walk. Why is it necessary?
4. If a stock has expected return 10% and volatility 30%, what is the per-period log-return drift in the GRW recursion?
5. Explain the Efficient Market Hypothesis and why it implies that stock prices cannot follow a mean-reverting process.
6. What is the long-run variance of an OU process with parameters $\kappa = 0.20$, $\sigma = 5$? What does this imply about long-horizon forecasts?
7. How would you estimate the parameters of an OU process from historical data? What regression do you run?
8. Why does the OU process autocorrelation decay exponentially?
9. Suppose you observe that an oil basis has been near zero for years with occasional brief spikes. What process would you use to model it? Why?

10. Why are jumps modeled as multiplicative for stocks (GRW) but additive for cash flows (ARW)?

11. If a process has a strong $\sqrt{t}$ rule for volatility growth, which family does it belong to: ARW/GRW or OU? Why?

12. Suppose you want to simulate corporate cash flows over 10 years. Which process would you choose, and what parameters would you estimate from historical data?

6.12 Numerical and Python Problems

Difficulty: S = Straightforward (mechanical), M = Moderate (combining ideas), C = Challenge (multi-step or open-ended).

1. [S] ARW Moments. Consider an ARW with $S_0 = 50$, $\mu = 2$, $\sigma = 8$, monthly. (a) Compute $\mathbb{E}[S_{12}]$, $\text{Var}(S_{12})$, and $\text{SD}(S_{12})$. (b) Compute a 95% confidence band at $t = 12$ months. (c) What is $\text{Cov}(S_6, S_{12})$?

2. [S] GRW Moments. A stock has $S_0 = \$100$, annual expected return 12%, annual volatility 25%. (a) Compute the expected price and variance of $\ln(S_T)$ at $T = 5$ years. (b) Compute the 5th and 95th percentiles of S_5. (c) Verify that $\mathbb{E}[S_5] = 100 \cdot e^{0.60} = 182.21$ matches your expected price.

3. [S] OU Forecast. An interest rate follows an OU process with $\mu = 4\%$, $\kappa = 0.10$ (monthly), $\sigma = 0.3\%$, and currently $r_0 = 7\%$. (a) Compute $\mathbb{E}[r_t]$ at $t = 6, 12, 60$ months. (b) Compute the long-run standard deviation. (c) At what time does the expected rate fall below 4.5%?

4. [M] Convexity Bias Demonstration. Simulate the GRW with $S_0 = 100$, $\alpha = 0.10$, $\sigma = 0.30$, $T = 1$ year and $N = 50{,}000$ paths. (a) Confirm that with the correction $(\alpha - \frac{1}{2}\sigma^2)$, $\mathbb{E}[S_T] \approx 100 \cdot e^{0.10} = 110.52$. (b) Remove the correction and confirm $\mathbb{E}[S_T]$ rises to about 115.6. (c) Plot the histogram of S_T for both cases.

5. [M] OU vs ARW: Variance Growth. Compare variance growth under ARW and OU. (a) For $\sigma = 5$, $\kappa = 0.20$, plot $\text{Var}(S_t)$ vs. t for $t = 1, 2, \ldots, 50$ under both processes. (b) At what horizon t^* does ARW variance exceed OU variance by 5×? (c) What does this say about modeling long-horizon variables?

6. [M] Parameter Estimation from Synthetic Data. Generate 300 monthly observations of an OU process with $\mu = 100$, $\kappa = 0.08$, $\sigma = 4$ and $S_0 = 120$. Use linear regression to estimate the parameters from your simulated data. (a) Report the estimates and their standard errors. (b) Test the null hypothesis of no mean reversion. (c) Repeat with sample sizes 100, 500, and 1000 and discuss how estimation quality scales with sample size.

7. [M] Real Data: S&P 500 Returns. Download daily S&P 500 closing prices for the most recent 10 years from Yahoo Finance or FRED (series SP500). (a) Compute daily log-returns. (b) Estimate α (annualized) and σ (annualized). (c) Test whether daily log-returns are normally distributed using a Q-Q plot and the Jarque-Bera test. Discuss the implications for the GRW model.

8. [M] Mean Reversion in Crude Oil. Download monthly West Texas Intermediate crude oil prices (FRED series WTISPLC) for the past 10 years. (a) Run the OU regression on the price series and report κ, μ, σ, and the t-statistic on mean reversion. (b) Run the same regression on log-prices. (c) Which specification fits better? Why might log-prices be a better choice for a commodity?

9. [C] GRW-with-Jumps Calibration. Simulate a GRW with jumps: $S_0 = 100$, $\alpha = 0.10$, $\sigma = 0.20$, jump probability 1% per month, log-jump size $N(-0.10, 0.10^2)$ (negatively skewed). Compare the distribution of 1-year returns with and without jumps. (a) Plot histograms. (b) Compute skewness and kurtosis of each. (c) Discuss how adding jumps captures features of real returns that a pure GRW misses.

10. [C] Multi-Factor Simulation. Simulate the joint evolution of (i) a stock following GRW with $\alpha = 0.10$, $\sigma = 0.25$ annual; (ii) an interest rate following OU with $\mu = 4\%$, $\kappa = 0.05$, $\sigma = 0.005$ monthly; (iii) inflation following OU with $\mu = 2.5\%$, $\kappa = 0.10$, $\sigma = 0.003$ monthly. Use a Cholesky decomposition to incorporate correlations of 0.3 between stock and rate, 0.5 between rate and inflation, and -0.2 between stock and inflation. Simulate 10,000 paths for 5 years. (a) Plot one sample path showing all three variables. (b) Compute the empirical correlation matrix and verify it matches the input. (c) What is the probability that inflation exceeds 4% at year 5?

Mini-Project 6.1: Stochastic Models for Three Financial Variables

Background. Real financial variables don't come with a label telling you which process to use. The skill of choosing the right process, and defending the choice, is essential for any quant career. This project asks you to perform this analysis for three variables and write a brief memo summarizing your conclusions.

Setup. Choose three financial variables, one each from the following categories:

- A **freely-traded asset price**: e.g., S&P 500 index, Apple stock, gold price.
- An **interest rate or basis**: e.g., 10-yr Treasury yield, 3-month T-bill rate, USD/EUR FX rate.
- A **commodity price**: e.g., WTI crude oil, natural gas, copper.

Download at least 10 years of monthly data from FRED (preferred) or Yahoo Finance. Use the most recent data possible.

Tasks.

- **1. Visual inspection.** Plot each time series. Document any visible features: trends, cyclicality, sudden jumps, mean-reverting behavior, regime changes.
- **2. Process selection.** For each variable, choose ARW, GRW, OU, or one of these plus jumps. Justify your choice using the Section 6.7 decision tree and observed patterns in the data.

- **3. Parameter estimation.** Estimate parameters using the methods of Section 6.5. Report all estimates with standard errors. For OU, include the t-test for mean reversion.
- **4. Validation.** Compare the empirical distribution of monthly changes (or log-returns) to the theoretical distribution implied by your model. Use a Q-Q plot or histogram overlay. Note any systematic deviations.
- **5. Forecast.** Use your calibrated model to generate 5-year forecast distributions. Plot the median path with 50% and 95% confidence bands.
- **6. Memo.** Write a 1-2 page memo summarizing your three model specifications, the justification for each, and your forecasts. Highlight any concerns about model adequacy.

Stretch goals (advanced students).

- (a) Estimate parameters separately for two distinct time periods (e.g., 2015-2019 vs 2021-2025). Are the parameters stable, or do they shift? What does this say about the model?
- (b) For at least one variable, fit a GRW-with-jumps model and compare to the pure base process. Does adding jumps improve fit?
- (c) Simulate the three variables jointly with a Cholesky-based correlation structure estimated from your data. Use the joint simulation to compute the probability that all three variables exceed their long-run means simultaneously at year 5.

Deliverable. Submit a Jupyter notebook containing all code, plots, and the memo. The memo should be readable by a non-technical audience while making clear that the underlying analysis is rigorous.

6.13 References and Further Reading

Bachelier, Louis (1900). *Théorie de la spéculation.* PhD thesis, École Normale Supérieure. The original mathematical model of stock prices and the first appearance of the random-walk idea in finance.

Ornstein, Leonard S. and George E. Uhlenbeck (1930). "On the Theory of the Brownian Motion." *Physical Review* 36(5): 823-841. The original paper introducing the mean-reverting process now bearing their names.

Hull, John C. (2022). *Options, Futures, and Other Derivatives*, 11th edition. Pearson. The standard practitioner reference for stochastic processes used in financial applications.

Tsay, Ruey S. (2010). *Analysis of Financial Time Series*, 3rd edition. Wiley. Excellent treatment of parameter estimation for financial time series, including the Dickey-Fuller test and more sophisticated alternatives.

Online resources:

- FRED economic data (St. Louis Fed): `fred.stlouisfed.org`
- Yahoo Finance historical data: `finance.yahoo.com`
- Federal Reserve H.15 daily interest rates: `federalreserve.gov/releases/h15`